

EDA 实验报告

实验题目： 十字路口交通灯控制及显示实验

学　　院： 自动化学院

班　　级： 电信 **2303** 班

指导教师：

学　　号：

姓　　名： 樊博文

一． 实验内容及要求

实验 3：十字路口交通灯控制及显示实验(易灵思 FPGA 开发板 T20F256+扩展板 VC-Study)

实验内容：修改教材 5.4.2（P.154）的十字路口交通灯控制电路，添加时钟使能模块及数码管显示模块，将原主干道的一组红黄绿灯改成左右两组红黄绿灯，将原支干道的一组红黄绿灯改成上下两组红黄绿灯，构成完整的十字路口交通灯控制及显示电路，

主干道及支干道的红黄绿灯用扩展板上彩色 LED 灯显示，定时输出用各自数码管显示，

拨动开关 SW1 用作传感器 S 的输入，

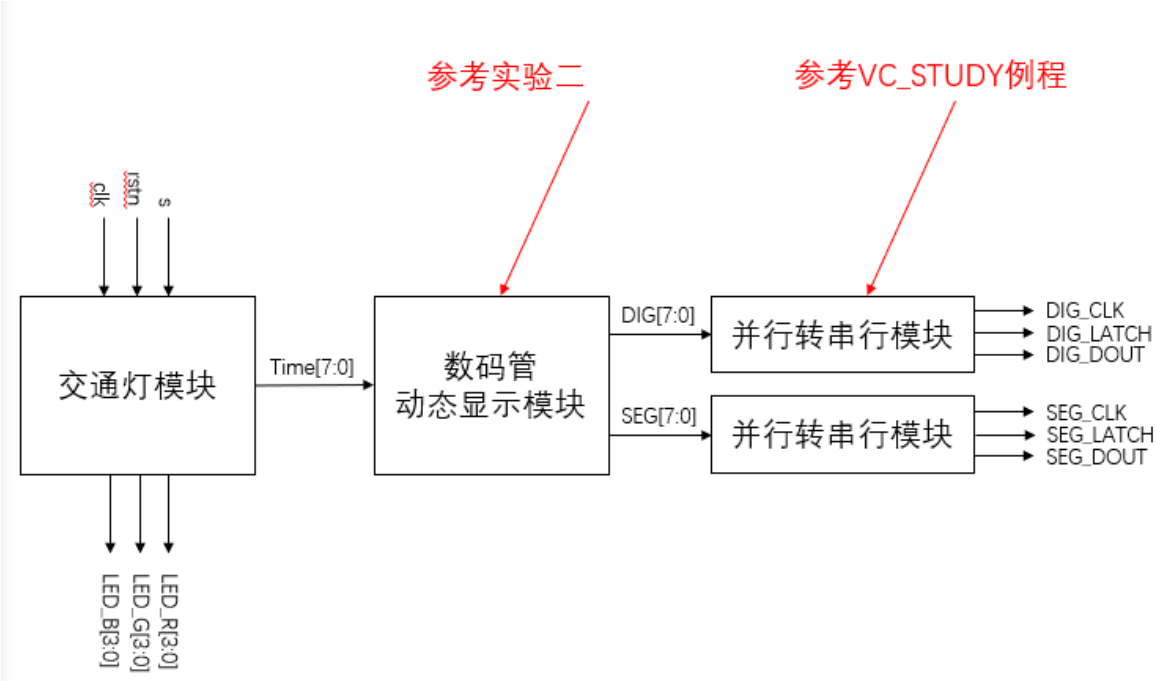
按键 1 用作复位。

实验步骤：

自主实验。

二． 实验原理及方法

1.系统总体结构



本实验采用“控制—显示—驱动”的分层结构：交通灯控制模块根据时钟、复位与传感器输入（SW1）产生当前相位的灯色控制信号与倒计时值；显示模块

将倒计时转换为数码管段码/位码；最后通过并行转串行电路把显示数据送到扩展板的 74HC595，从而驱动数码管显示。该思路与实验要求中“增加时钟使能与数码管显示、彩色 LED 显示灯色、SW1 作传感器、按键作复位”的目标一致。

左侧交通灯模块输入 clk（系统时钟）、rst_n（复位）以及 S（支路信号），输出两类结果：灯色控制信号 LED_R[3:0]、LED_G[3:0]、LED_B[3:0]，用于驱动扩展板彩色 LED；倒计时数据 Time[7:0]，表示当前相位剩余秒数。

中间数码管动态显示模块将 Time[7:0] 转换为数码管显示所需的两路并行信号：DIG[7:0]：位选信号和 SEG[7:0]段选信号。

右侧并行转串行模块，由于扩展板数码管通常通过 74HC595 级联驱动，位选与段选分别经过两路并行转串行模块，输出 *_CLK、*_LATCH、*_DOUT 三线串行接口信号连接扩展板。

2.实验原理

2.1 时钟使能与倒计时产生

FPGA 板载系统时钟为 50MHz。为获得“秒级”的时间基准，设计中采用计数器 timer_cnt 对时钟进行分频：当计数达到 CNT_1S_MAX = 50,000,000 对应的终值时，产生一个仅持续 1 个时钟周期的脉冲信号 time_en，作为 1s 时钟使能。其工作机制为：

当 timer_cnt == CNT_1S_MAX - 1 时，time_en = 1；其余时间 time_en = 0；倒计时寄存器 keep_time 仅在 time_en 有效时更新（通常为 keep_time <= keep_time - 1），从而实现严格的“每秒递减一次”，保证计时与真实时间一致。在显示数据组织上，顶层将倒计时值拼接为 w_seg_data = {15'd0, keep_time}，即把 keep_time 放在显示数据低位，统一交由后级数码管动态显示模块进行译码与扫描显示。这样既保持数据通道简单，也便于后续扩展更多显示内容。

在此秒级时间基准之上，系统将倒计时递减、状态保持与相位切换均限定在 time_en 有效的节拍边界内执行，从而保证控制逻辑与显示更新同步稳定。如图给出了交通灯控制器在每个 1s 节拍到来时的主流程与分支处理顺序。

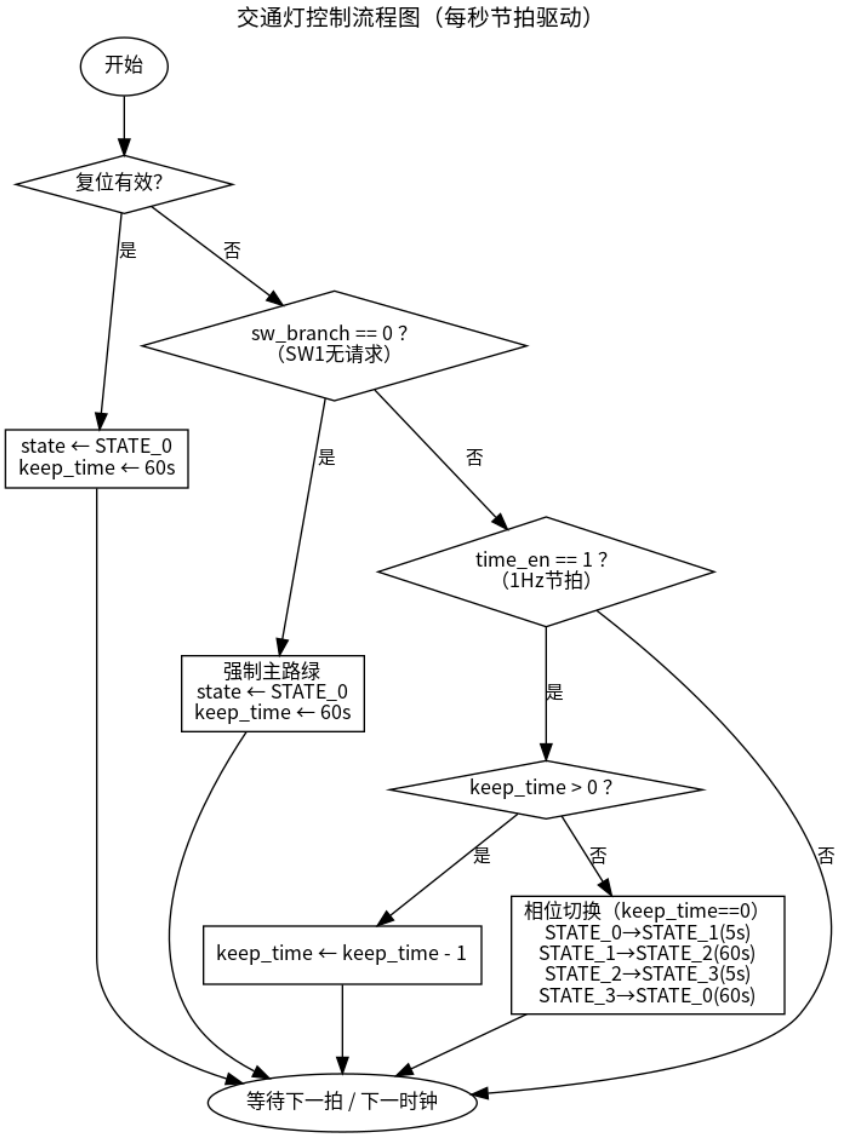


图-控制按秒节拍驱动主流程

如图所示，控制器在每个 1s 节拍到来时按固定优先级执行更新：首先判断复位，复位有效时将状态初始化并装载初始相位时长；复位无效时，再判断支路请求信号 sw_branch，当无请求时强制保持/回到主路放行相位并重置计时；当存在请求时，根据 keep_time 是否归零决定执行“倒计时递减”或“相位切换并装载下一相位时长”。通过将更新动作严格限定在节拍边界，避免了倒计时与相位切换在一个周期内发生多次变化，从而保证显示与控制行为稳定一致。

2.2 交通灯控制状态机逻辑

交通灯控制采用四状态有限状态机（FSM），用 STATE_0 ~ STATE_3 表示典型十字路口的“通行—过渡—通行—过渡”相位循环：

STATE_0：主干道绿、支干道红（主通行）

STATE_1：主干道黄、支干道红（主黄缓冲）

STATE_2：主干道红、支干道绿（支通行）

STATE_3：主干道红、支干道黄（支黄缓冲）

各相位持续时间由参数统一设定：主绿/支红相位为 TIME_GREEN_RED = 60s，黄灯缓冲相位为 TIME_YELLOW = 5s。状态机在 time_en 的 1s 节拍下驱动 keep_time 逐秒递减，当 keep_time 递减至 0 时，切换到下一状态并装载下一相位的预置时间，实现“相位切换与倒计时联动”。

同时，系统引入 SW1 作为支路车辆检测/请求输入，形成“主路优先、支路按需放行”的控制策略：当 sw_branch == 0（SW1 未触发，支路无请求）时，状态机强制保持/回到 STATE_0，并将 keep_time 重置为 60s，相当于主干道持续绿灯通行；当 sw_branch == 1（检测到支路需求）时，状态机才按照既定顺序运行，并在倒计时归零时进入下一相位。

该机制符合实际路口控制规律：在支路无车时避免频繁切灯，提高主路通行效率；在支路有请求时能够及时响应并完成完整的过渡相位，保证通行安全。

2.3 彩色 LED 显示原理与方向映射

实验中交通灯灯色通过扩展板 RGB 彩灯显示。该 RGB 灯为共阳极接法（“0 亮 1 灭”），因此控制输出采用低电平点亮的方式。为简化实现，设计中将蓝色通道固定关闭（LED_B = 4'b1111），仅使用红、绿两色组合实现三种灯色：

红灯：红亮、绿灭

绿灯：绿亮、红灭

黄灯：红与绿同时点亮（叠加呈黄色效果）

方向映射方面，主路与支路分别用两组 2bit 信号描述（main_R/main_G 与 side_R/side_G），并映射到 4 颗 RGB 灯：LED[0]、LED[2] 表示主干道左右两个方向；LED[1]、LED[3] 表示支干道上下两个方向。

结合实物运行现象可见：彩灯会随状态机在“主绿/主黄/支绿/支黄”等相位间切换；同时数码管显示当前相位剩余秒数（如 59、08、06 等），用于直观验证 keep_time 的倒计时递减与相位切换是否同步。

2.4 数码管显示原理

数码管显示链路按数据转换—动态扫描—串行输出三步实现：

1.二进制到十进制位（8421 BCD 转换）

keep_time 为二进制计数值。为在数码管上显示十进制，需要将其转换为个位、十位等十进制位。系统通过 bcd_8421 模块将输入数据转换为 unit/ten/hun/tho/... 等 BCD 码，为后续段码译码提供基础。

2.动态扫描显示（seg_dynamic）

为节省 IO 并保证显示稳定，采用动态扫描方式：通过毫秒级刷新计数，轮流选通不同数码管位(sel[7:0] 位选),同时输出对应的段码(seg[7:0] 段选)。借助人眼视觉暂留效果，实现“多位同时点亮”的显示观感。

3.74HC595 并行转串行输出（hc595_ctrl）

扩展板数码管由 74HC595 级联驱动，因此系统使用两路 hc595_ctrl 分别传输：

位选链路：w_sel → DIG_DOUT / DIG_CLK / DIG_LATCH

段选链路：w_seg → DP_DOUT / DP_CLK / DP_LATCH

通过移位时钟与锁存信号将并行的位/段数据稳定输出到数码管端，实现持续刷新显示。因此，倒计时数据会在动态扫描与 595 串行驱动的配合下持续更新，使数码管能够与交通灯状态同步展示每个相位的剩余时间。

3.实验方法

搭建系统总体结构，按控制—显示—驱动分层完成顶层连接。

完成秒级时钟使能与倒计时电路，实现 1 s 递减节拍。

设计交通灯四状态 FSM，并设置各相位持续时间参数。

引入 SW1 作为支路传感器输入，完成主路优先、按需放行控制策略。

完成数码管显示链路（BCD/动态扫描/74HC595 串行驱动），实现倒计时显示。进行功能仿真验证，通过后上板联调并记录实验现象。

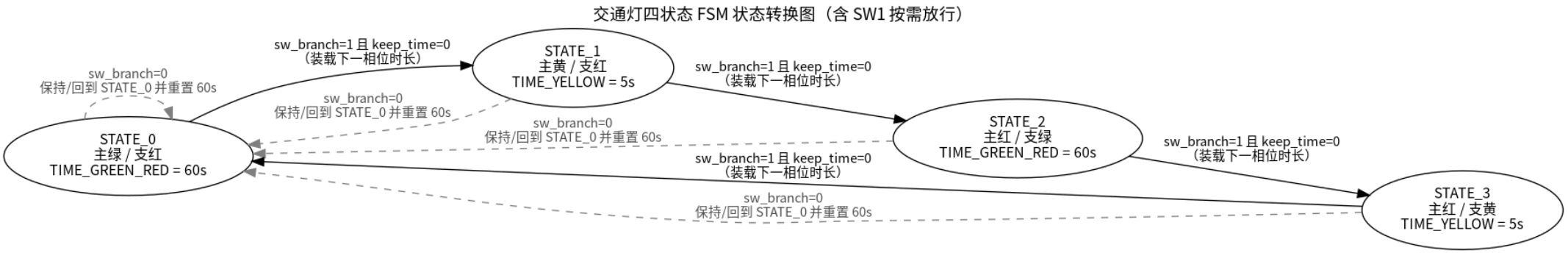


图-交通灯状态转换

如图所示，控制流程以 1 Hz 节拍信号 time_en 为驱动，确保所有计时与相位更新均在“秒边界”发生，避免出现亚秒级抖动或多次切换。整体逻辑分为三类分支：

1）复位分支：当复位有效时，系统将 state 初始化为 STATE_0，并装载 keep_time=60s，保证系统从主路优先放行的初始相位开始运行。

2）无请求分支：当检测到 sw_branch==0（SW1 无请求）时，控制器强制保持/回到 STATE_0，并将 keep_time 重置为 60s，使主路持续放行，体现“主路优先”的策略。

3）正常计时与切换分支：当 sw_branch==1 且 time_en==1 时，若 keep_time>0，则执行 keep_time←keep_time-1 完成倒计时递减；当 keep_time==0 时触发相位切换，进入下一状态并装载下一相位时长。完成本拍处理后进入等待，直至下一次 time_en 到来。

三．代码设计（说明修改优化部分）

1.交通灯与倒计时综合控制

```
`timescale 1ns/1ns

module rbg_led_test
(
    input               pll_inst1_CLKOUT0, //  时钟输入
    input               key_rst,                    //  复位输入

    input               sw_branch,              //  支路控制开关

    output reg [3:0] LED_R,
    output reg [3:0] LED_G,
    output reg [3:0] LED_B,

    output   wire     DIG_CLK,          // shcp
    output   wire     DIG_LATCH,        // stcp
    output   wire     DIG_DOUT,         // ds
    output   wire     DP_CLK,           // shcp
    output   wire     DP_LATCH,         // stcp
    output   wire     DP_DOUT             // ds
);



    wire sys_clk = pll_inst1_CLKOUT0;
    wire sys_rst_n = key_rst;


    //  红绿灯核心逻辑
    parameter CNT_1S_MAX = 50_000_000;
    localparam STATE_0 = 0;
    localparam STATE_1 = 1;
    localparam STATE_2 = 2;
    localparam STATE_3 = 3;

    localparam TIME_GREEN_RED = 60 ;
    localparam TIME_YELLOW      = 5;

    reg [25:0] timer_cnt;
    wire           time_en;
    reg [1:0]   state;
```

```
    reg [5:0]    keep_time;

    // 1 秒计数器
    always @(posedge sys_clk or negedge sys_rst_n) begin
        if (!sys_rst_n) timer_cnt <= 0;
        else if (timer_cnt >= CNT_1S_MAX - 1) timer_cnt <= 0;
        else timer_cnt <= timer_cnt + 1'b1;
    end
    assign time_en = (timer_cnt == CNT_1S_MAX - 1);

    // 状态机
    always @(posedge sys_clk or negedge sys_rst_n) begin
        if (!sys_rst_n) begin
            state <= STATE_0;
            keep_time <= TIME_GREEN_RED;
        end
        else begin
            // 关键逻辑：开关没开(0) -> 强制主绿(STATE_0)
            if (!sw_branch) begin
                state <= STATE_0;
                keep_time <= TIME_GREEN_RED;
            end
            else if (time_en) begin
                if (keep_time > 0)
                    keep_time <= keep_time - 1;
                else begin
                    case (state)
                        STATE_0: begin state <= STATE_1; keep_time <= TIME_YELLOW;
end
                        STATE_1: begin state <= STATE_2; keep_time <= TIME_GREEN_RED;
end
                        STATE_2: begin state <= STATE_3; keep_time <= TIME_YELLOW;
end
                        STATE_3: begin state <= STATE_0; keep_time <= TIME_GREEN_RED;
end
                    endcase
                end
            end
        end
    end

    // LED 输出逻辑 (共阳极: 0 亮 1 灭)
    reg [1:0] main_R, main_G;
    reg [1:0] side_R, side_G;
```

```
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        // 复位时，全灭 (1111)
        LED_R <= 4'b1111; LED_G <= 4'b1111; LED_B <= 4'b1111;
        main_R <= 2'b11; main_G <= 2'b11;
        side_R <= 2'b11; side_G <= 2'b11;
    end
    else begin
        LED_B <= 4'b1111; // 蓝灯常灭

        case (state)
            STATE_0: begin // 主绿 支红
                main_R <= 2'b11; main_G <= 2'b00;
                side_R <= 2'b00; side_G <= 2'b11;
            end
            STATE_1: begin // 主黄 支红
                main_R <= 2'b00; main_G <= 2'b00;
                side_R <= 2'b00; side_G <= 2'b11;
            end
            STATE_2: begin // 主红 支绿
                main_R <= 2'b00; main_G <= 2'b11;
                side_R <= 2'b11; side_G <= 2'b00;
            end
            STATE_3: begin // 主红 支黄
                main_R <= 2'b00; main_G <= 2'b11;
                side_R <= 2'b00; side_G <= 2'b00;
            end
        endcase

        // 物理映射: 0,2 主路; 1,3 支路
        LED_R[0] <= main_R[0]; LED_G[0] <= main_G[0];
        LED_R[2] <= main_R[1]; LED_G[2] <= main_G[1];
        LED_R[1] <= side_R[0]; LED_G[1] <= side_G[0];
        LED_R[3] <= side_R[1]; LED_G[3] <= side_G[1];
    end
end

wire [19:0] w_seg_data;
wire [7:0]    w_sel;
wire [7:0]    w_seg;

// 显示倒计时
assign w_seg_data = {15'd0, keep_time};
```

```
    // 数码管解码
    seg_dynamic u_seg_decode
    (
        .sys_clk        (sys_clk),
        .sys_rst_n      (sys_rst_n),
        .data           (w_seg_data),
        .point          (8'b0),
        .seg_en         (1'b1),
        .sign           (1'b0),
        .sel            (w_sel),
        .seg            (w_seg)
    );

    // 第一路 595 (位选)
    hc595_ctrl   u1_hc595_dig (
        .sys_clk        (sys_clk),
        .sys_rst_n      (sys_rst_n),
        .seg_data       (w_sel),
        .stcp           (DIG_LATCH),
        .shcp           (DIG_CLK),
        .ds             (DIG_DOUT)
    );

    // 第二路 595 (段选)
    hc595_ctrl   u2_hc595_dp (
        .sys_clk        (sys_clk),
        .sys_rst_n      (sys_rst_n),
        .seg_data       (w_seg),
        .stcp           (DP_LATCH),
        .shcp           (DP_CLK),
        .ds             (DP_DOUT)
    );

endmodule
```

2.数码管动态显示

```
//seg_dynamic.v
`timescale   1ns/1ns
// Description    : 数码管动态显示
module   seg_dynamic
(
    input    wire                sys_clk       , //系统时钟，频率 50MHz
```

```
    input   wire            sys_rst_n   ,//复位信号，低有效
    input   wire    [19:0]  data        ,//数码管要显示的值
    input   wire    [7:0]   point       ,//小数点显示,高电平有效
    input   wire            seg_en      ,//数码管使能信号，高电平有效
    input   wire            sign        ,//符号位，高电平显示负号

    output  reg     [7:0]   sel         ,//数码管位选信号
    output  reg     [7:0]   seg           //数码管段选信号
);

//parameter define
parameter   CNT_MAX =   16'd49_999;  //数码管刷新时间计数最大值

//wire  define
wire    [3:0]   unit        ;   //个位数
wire    [3:0]   ten         ;   //十位数
wire    [3:0]   hun         ;   //百位数
wire    [3:0]   tho         ;   //千位数
wire    [3:0]   t_tho       ;   //万位数
wire    [3:0]   h_hun       ;   //十万位数

//reg   define
reg     [23:0]  data_reg    ;   //待显示数据寄存器
reg     [15:0]  cnt_1ms     ;   //1ms 计数器
reg             flag_1ms    ;   //1ms 标志信号
reg     [2:0]   cnt_sel     ;   //数码管位选计数器
reg     [7:0]   sel_reg     ;   //位选信号
reg     [3:0]   data_disp   ;   //当前数码管显示的数据
reg             dot_disp    ;   //当前数码管显示的小数点

//data_reg：控制数码管显示数据
 always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        data_reg    <=  24'b0;
//若显示的十进制数的十万位为非零数据或需显示小数点，则六个数码管全显示
    else    if((h_hun) || (point[5]))
        data_reg    <=  {h_hun,t_tho,tho,hun,ten,unit};
//若显示的十进制数的万位为非零数据或需显示小数点，则值显示在 5 个数码管上
//打比方我们输入的十进制数据为 20’d12345，我们就让数码管显示 12345 而不是 012345
    else    if(((t_tho) || (point[4])) && (sign == 1'b1))//显示负号
        data_reg <= {4'd10,t_tho,tho,hun,ten,unit};//4'd10 我们定义为显示负号
    else    if(((t_tho) || (point[4])) && (sign == 1'b0))
        data_reg <= {4'd11,t_tho,tho,hun,ten,unit};//4'd11 我们定义为不显示
//若显示的十进制数的千位为非零数据或需显示小数点，则值显示 4 个数码管
```

```
        else    if(((tho) || (point[3])) && (sign == 1'b1))
                data_reg <= {4'd11,4'd10,tho,hun,ten,unit};
        else    if(((tho) || (point[3])) && (sign == 1'b0))
                data_reg <= {4'd11,4'd11,tho,hun,ten,unit};
//若显示的十进制数的百位为非零数据或需显示小数点，则值显示 3 个数码管
        else    if(((hun) || (point[2])) && (sign == 1'b1))
                data_reg <= {4'd11,4'd11,4'd10,hun,ten,unit};
        else    if(((hun) || (point[2])) && (sign == 1'b0))
                data_reg <= {4'd11,4'd11,4'd11,hun,ten,unit};
//若显示的十进制数的十位为非零数据或需显示小数点，则值显示 2 个数码管
        else    if(((ten) || (point[1])) && (sign == 1'b1))
                data_reg <= {4'd11,4'd11,4'd11,4'd10,ten,unit};
        else    if(((ten) || (point[1])) && (sign == 1'b0))
                data_reg <= {4'd11,4'd11,4'd11,4'd11,ten,unit};
//若显示的十进制数的个位且需显示负号
        else    if(((unit) || (point[0])) && (sign == 1'b1))
                data_reg <= {4'd11,4'd11,4'd11,4'd11,4'd10,unit};
//若上面都不满足都只显示一位数码管
        else
                data_reg <= {4'd11,4'd11,4'd11,4'd11,4'd11,unit};

//cnt_1ms:1ms 循环计数
always@(posedge sys_clk or negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
                cnt_1ms <=    16'd0;
        else    if(cnt_1ms == CNT_MAX)
                cnt_1ms <=    16'd0;
        else
                cnt_1ms <=    cnt_1ms + 1'b1;

//flag_1ms:1ms 标志信号
always@(posedge sys_clk or negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
                flag_1ms      <=    1'b0;
        else    if(cnt_1ms == CNT_MAX - 1'b1)
                flag_1ms      <=    1'b1;
        else
                flag_1ms      <=    1'b0;

//cnt_sel：从 0 到 7 循环数，用于选择当前显示的数码管
always@(posedge sys_clk or negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
                cnt_sel <=    3'd0;
        else    if((cnt_sel == 3'd7) && (flag_1ms == 1'b1))
```

```
            cnt_sel <=    3'd0;
        else        if(flag_1ms == 1'b1)
            cnt_sel <=    cnt_sel + 1'b1;
        else
            cnt_sel <=    cnt_sel;

//数码管位选信号寄存器
always@(posedge sys_clk or negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
            sel_reg <=    8'b00000_000;
        else        if((cnt_sel == 3'd0) && (flag_1ms == 1'b1))
            sel_reg <=    8'b00000_001;
        else        if(flag_1ms == 1'b1)
            sel_reg <=    sel_reg << 1;
        else
            sel_reg <=    sel_reg;

//控制数码管的位选信号，使六个数码管轮流显示
always@(posedge sys_clk or    negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
            data_disp       <=    8'b0;
        else        if((seg_en == 1'b1) && (flag_1ms == 1'b1))
            case(cnt_sel)
            3'd0:     data_disp       <=    data_reg[3:0];
            3'd1:     data_disp       <=    data_reg[3:0];
            3'd2:     data_disp       <=    data_reg[3:0];
            3'd3:     data_disp       <=    data_reg[3:0];
            3'd4:     data_disp       <=    data_reg[7:4];
            3'd5:     data_disp       <=    data_reg[7:4];
            3'd6:     data_disp       <=    data_reg[7:4];
            3'd7:     data_disp       <=    data_reg[7:4];
            default:data_disp       <=    8'b0            ;
            endcase
        else
            data_disp     <=    data_disp;

//dot_disp：小数点低电平点亮，需对小数点有效信号取反
always@(posedge sys_clk or negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
            dot_disp       <=    1'b1;
        else        if(flag_1ms == 1'b1)
            dot_disp       <=    ~point[cnt_sel];
        else
            dot_disp       <=    dot_disp;
```

```
//控制数码管段选信号，显示数字
always@(posedge sys_clk or    negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        seg <=   8'b1111_1111;
    else
        case(data_disp)
            4'd0   : seg   <= 8'b0000_0011 ;      //显示数字 0
            4'd1   : seg   <= 8'b1001_1111 ;      //显示数字 1
            4'd2   : seg   <= 8'b0010_0101 ;      //显示数字 2
            4'd3   : seg   <= 8'b0000_1101 ;      //显示数字 3
            4'd4   : seg   <= 8'b1001_1001 ;      //显示数字 4
            4'd5   : seg   <= 8'b0100_1001 ;      //显示数字 5
            4'd6   : seg   <= 8'b0100_0001 ;      //显示数字 6
            4'd7   : seg   <= 8'b00011111 ;      //显示数字 7
            4'd8   : seg   <= 8'b00000001 ;      //显示数字 8
            4'd9   : seg   <= 8'b00011001 ;      //显示数字 9
            4'd10 : seg   <=   8'b1011_1111              ;      //显示负号
            4'd11 : seg   <=   8'b1111_1111              ;      //不显示任何字符
            default:seg   <=   8'b1100_0000;
        endcase

//sel:数码管位选信号赋值
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        sel <=   8'b00000000;
    else
        sel <=   sel_reg;
//---------- bsd_8421_inst ----------
bcd_8421       bcd_8421_inst
(
    .sys_clk        (sys_clk   ),     //系统时钟，频率 50MHz
    .sys_rst_n      (sys_rst_n),     //复位信号，低电平有效
    .data           (data       ),     //输入需要转换的数据

    .unit           (unit       ),     //个位 BCD 码
    .ten            (ten        ),     //十位 BCD 码
    .hun            (hun        ),     //百位 BCD 码
    .tho            (tho        ),     //千位 BCD 码
    .t_tho          (t_tho      ),     //万位 BCD 码
    .h_hun          (h_hun      )      //十万位 BCD 码
);

endmodule
```

3.74HC595 控制模块

```
`timescale    1ns/1ns
module    hc595_ctrl
(
      input     wire                sys_clk       ,    //系统时钟，频率 50MHz
      input     wire                sys_rst_n     ,    //复位信号，低有效
      // input     wire      [5:0]     sel           ,    //数码管位选信号
      input     wire      [7:0]     seg_data            ,    //数码管段选信号

      output    reg                 stcp          ,    //数据存储器时钟
      output    reg                 shcp          ,    //移位寄存器时钟
      output    reg                 ds                 //串行数据输入
);
//reg     define
reg       [1:0]     cnt_4     ;     //分频计数器
reg       [3:0]     cnt_bit ;     //传输位数计数器

//wire    define
wire      [13:0]    data      ;     //数码管信号寄存

parameter SERDATA = 8;

//将数码管信号寄存
assign    data = seg_data;


//分频计数器:0~3 循环计数
always@(posedge sys_clk or    negedge sys_rst_n)
      if(sys_rst_n == 1'b0)
            cnt_4 <=    2'd0;
      else      if(cnt_4 == 2'd3)
            cnt_4 <=    2'd0;
      else
            cnt_4 <=    cnt_4 +     1'b1;

//cnt_bit:每输入一位数据加一
always@(posedge sys_clk or    negedge sys_rst_n)
      if(sys_rst_n == 1'b0)
            cnt_bit     <=    4'd0;
      else      if(cnt_4 == 2'd3 && cnt_bit == SERDATA-1)
            cnt_bit     <=    4'd0;
      else      if(cnt_4    ==    2'd3)
```

```
            cnt_bit     <=    cnt_bit     +     1'b1;
        else
            cnt_bit     <=    cnt_bit;

//stcp:14 个信号传输完成之后产生一个上升沿
always@(posedge sys_clk or    negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
            stcp        <=    1'b0;
        else        if(cnt_bit == SERDATA- 1 && cnt_4 == 2'd3)
            stcp        <=    1'b1;
        else
            stcp        <=    1'b0;

//shcp:产生四分频移位时钟
always@(posedge sys_clk or    negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
            shcp        <=    1'b0;
        else        if(cnt_4 >= 4'd2)
            shcp        <=    1'b1;
        else
            shcp        <=    1'b0;

//ds:将寄存器里存储的数码管信号输入即
always@(posedge sys_clk or    negedge sys_rst_n)
        if(sys_rst_n == 1'b0)
            ds    <=    1'b0;
        else        if(cnt_4 == 2'd0)
            ds    <=    data[cnt_bit];
        else
            ds    <=    ds;

endmodule
```

4.二进制转 BCD 转换

```
`timescale    1ns/1ns
// Description     : 二进制数转 BCD 码
module    bcd_8421
(
        input     wire                  sys_clk        ,    //系统时钟，频率 50MHz
        input     wire                  sys_rst_n      ,    //复位信号，低电平有效
        input     wire       [19:0]     data           ,    //输入需要转换的数据

        output    reg        [3:0]      unit           ,    //个位 BCD 码
```

```
    output   reg      [3:0]   ten          ,   //十位 BCD 码
    output   reg      [3:0]   hun          ,   //百位 BCD 码
    output   reg      [3:0]   tho          ,   //千位 BCD 码
    output   reg      [3:0]   t_tho        ,   //万位 BCD 码
    output   reg      [3:0]   h_hun            //十万位 BCD 码
);
//reg    define
reg      [4:0]   cnt_shift   ;   //移位判断计数器
reg      [43:0]  data_shift  ;   //移位判断数据寄存器
reg              shift_flag  ;   //移位判断标志信号
//cnt_shift:从 0 到 21 循环计数
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_shift   <=  5'd0;
    else    if((cnt_shift == 5'd21) && (shift_flag == 1'b1))
        cnt_shift   <=  5'd0;
    else    if(shift_flag == 1'b1)
        cnt_shift   <=  cnt_shift + 1'b1;
    else
        cnt_shift   <=  cnt_shift;

//data_shift：计数器为 0 时赋初值，计数器为 1~20 时进行移位判断操作
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        data_shift  <=  44'b0;
    else    if(cnt_shift == 5'd0)
        data_shift  <=  {24'b0,data};
    else    if((cnt_shift <= 20) && (shift_flag == 1'b0))
        begin
            data_shift[23:20]   <=  (data_shift[23:20] > 4) ? (data_shift[23:20] + 2'd3) :
(data_shift[23:20]);
            data_shift[27:24]   <=  (data_shift[27:24] > 4) ? (data_shift[27:24] + 2'd3) :
(data_shift[27:24]);
            data_shift[31:28]   <=  (data_shift[31:28] > 4) ? (data_shift[31:28] + 2'd3) :
(data_shift[31:28]);
            data_shift[35:32]   <=  (data_shift[35:32] > 4) ? (data_shift[35:32] + 2'd3) :
(data_shift[35:32]);
            data_shift[39:36]   <=  (data_shift[39:36] > 4) ? (data_shift[39:36] + 2'd3) :
(data_shift[39:36]);
            data_shift[43:40]   <=  (data_shift[43:40] > 4) ? (data_shift[43:40] + 2'd3) :
(data_shift[43:40]);
        end
    else    if((cnt_shift <= 20) && (shift_flag == 1'b1))
        data_shift  <=  data_shift << 1;
```

```
    else
        data_shift   <=   data_shift;

//shift_flag：移位判断标志信号，用于控制移位判断的先后顺序
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        shift_flag   <=   1'b0;
    else
        shift_flag   <=   ~shift_flag;

//当计数器等于 20 时，移位判断操作完成，对各个位数的 BCD 码进行赋值
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        begin
            unit      <=   4'b0;
            ten       <=   4'b0;
            hun       <=   4'b0;
            tho       <=   4'b0;
            t_tho     <=   4'b0;
            h_hun     <=   4'b0;
        end
    else      if(cnt_shift == 5'd21)
        begin
            unit      <=   data_shift[23:20];
            ten       <=   data_shift[27:24];
            hun       <=   data_shift[31:28];
            tho       <=   data_shift[35:32];
            t_tho     <=   data_shift[39:36];
            h_hun     <=   data_shift[43:40];
        end

endmodule
```

5.数码管和 595 显示模块

```
`timescale   1ns/1ns
// Description    ：数码管动态显示
module   seg_595_dynamic
(
    input    wire              sys_clk      ,//系统时钟，频率 50MHz
    input    wire              sys_rst_n    ,//复位信号，低有效
    input    wire    [19:0]    data         ,//数码管要显示的值
    input    wire    [5:0]     point        ,//小数点显示,高电平有效
    input    wire              seg_en       ,//数码管使能信号，高电平有效
    input    wire              sign         ,//符号位，高电平显示负号
```

```
    output    wire                stcp          ,//数据存储器时钟
    output    wire                shcp          ,//移位寄存器时钟
    output    wire                ds            ,//串行数据输入
    output    wire                oe              //使能信号

);
//wire    define
wire      [5:0]     sel;      //数码管位选信号
wire      [7:0]     seg;      //数码管段选信号
//---------- seg_dynamic_inst ----------
seg_dynamic seg_dynamic_inst
(
    .sys_clk      (sys_clk    ),    //系统时钟，频率 50MHz
    .sys_rst_n    (sys_rst_n),    //复位信号，低有效
    .data         (data       ),    //数码管要显示的值
    .point        (point      ),    //小数点显示,高电平有效
    .seg_en       (seg_en     ),    //数码管使能信号，高电平有效
    .sign         (sign       ),    //符号位，高电平显示负号

    .sel          (sel        ),    //数码管位选信号
    .seg          (seg        )     //数码管段选信号

);

//---------- hc595_ctrl_inst ----------
hc595_ctrl    hc595_ctrl_inst
(
    .sys_clk      (sys_clk    ),    //系统时钟，频率 50MHz
    .sys_rst_n    (sys_rst_n),    //复位信号，低有效
    .sel          (sel        ),    //数码管位选信号
    .seg          (seg        ),    //数码管段选信号

    .stcp         (stcp       ),    //输出数据存储寄时钟
    .shcp         (shcp       ),    //移位寄存器的时钟输入
    .ds           (ds         ),    //串行数据输入
    .oe           (oe         )

);

Endmodule


`timescale    1ns/1ns
```

```
module    seg_595_static
(
      input                                    rst_n                          ,
      input                                    pll_inst1_CLKOUT0              ,//50Mhz
      output                                   DIG_CLK                        ,//shcp
      output                                   DIG_LATCH                      ,//stcp
      output                                   DIG_DOUT                       ,
      output                                   DP_CLK                         ,//shcp
      output                                   DP_LATCH                       ,//stcp
      output                                   DP_DOUT
);
// data_generator

wire      [19:0]    data      ;    //数码管要显示的值
wire      [7:0]     point     ;    //小数点显示,高电平有效 top_seg_595
wire                seg_en    ;    //数码管使能信号，高电平有效
wire                sign      ;    //符号位，高电平显示负号

data_gen      data_gen_inst
(
      .sys_clk        (pll_inst1_CLKOUT0    ),    //系统时钟，频率 50MHz
      .sys_rst_n      (rst_n),    //复位信号，低电平有效

      .data           (data         ),    //数码管要显示的值
      .point          (point        ),    //小数点显示,高电平有效
      .seg_en         (seg_en       ),    //数码管使能信号，高电平有效
      .sign           (sign         )     //符号位，高电平显示负号
);

wire [7:0] sel;
wire [7:0] seg;

// select seg
seg_dynamic seg_dynamic_inst
(
      .sys_clk        (pll_inst1_CLKOUT0    ),    //系统时钟，频率 50MHz
      .sys_rst_n      (rst_n),    //复位信号，低有效
      .data           (data         ),    //数码管要显示的值
      .point          (point        ),    //小数点显示,高电平有效
      .seg_en         (seg_en       ),    //数码管使能信号，高电平有效
      .sign           (sign         ),    //符号位，高电平显示负号

      .sel            (sel          ),    //数码管位选信号
      .seg            (seg          )     //数码管段选信号
```

```
);
hc595_ctrl    u1_hc595_ctrl (
      .sys_clk                    ( pll_inst1_CLKOUT0              ),
      .sys_rst_n                  ( rst_n                          ),
      .seg_data                   ( sel     [7:0]                  ),

      .stcp                       ( DIG_LATCH                      ),
      .shcp                       ( DIG_CLK                        ),
      .ds                         ( DIG_DOUT                       )
);
hc595_ctrl    u2_hc595_ctrl (
      .sys_clk                    ( pll_inst1_CLKOUT0    ),
      .sys_rst_n                  ( rst_n                          ),
      .seg_data                   ( seg     [7:0]                  ),

      .stcp                       (     DP_LATCH                   ),
      .shcp                       (DP_CLK                ),
      .ds                         (DP_DOUT                         )
);
endmodule
```

6. 数码管显示数据生成

```
`timescale    1ns/1ns
// Description     : 生成数码管显示数据
module    data_gen
#(
      parameter     CNT_MAX = 26'd25_000_000, //1s 计数值
      parameter     DATA_MAX= 'd60     //显示的最大值
)
(
      input     wire                sys_clk       ,    //系统时钟，频率 50MHz
      input     wire                sys_rst_n     ,    //复位信号，低电平有效

      output    reg       [19:0]    data          ,    //数码管要显示的值
      output    wire      [7:0]     point         ,    //小数点显示,高电平有效
      output    reg                 seg_en        ,    //数码管使能信号，高电平有效
      output    wire                sign                //符号位，高电平显示负号
);


//reg     define
reg       [25:0]    cnt_100ms     ;     //100ms 计数器
```

```
reg                 cnt_flag    ;   //100ms 标志信号

//不显示小数点以及负数
assign  point   =   8'b0000_0000;
assign  sign    =   1'b0;

//cnt_100ms:用 50MHz 时钟从 0 到 4999_999 计数即为 100ms
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_100ms   <=  26'd0;
    else    if(cnt_100ms == CNT_MAX)
        cnt_100ms   <=  26'd0;
    else
        cnt_100ms   <=  cnt_100ms + 1'b1;

//cnt_flag:每 100ms 产生一个标志信号
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_flag    <=  1'b0;
    else    if(cnt_100ms == CNT_MAX - 1'b1)
        cnt_flag    <=  1'b1;
    else
        cnt_flag    <=  1'b0;

//数码管显示的数据:0-999_999
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        data    <=  20'd0;
    else    if((data == DATA_MAX) && (cnt_flag == 1'b1))
        data    <=  20'd0;
    else    if(cnt_flag == 1'b1)
        data    <=  data + 1'b1;
    else
        data    <=  data;

//数码管使能信号给高即可
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        seg_en  <=  1'b0;
    else
        seg_en  <=  1'b1;

endmodule
```

7.数码管显示顶层测试

```
`timescale    1ns/1ns
// Description     : 数码管动态显示，顶层模块
module    top_seg_595
(
      input     wire                sys_clk       ,    //系统时钟，频率 50MHz
      input     wire                sys_rst_n     ,    //复位信号，低电平有效

      output    wire                stcp          ,    //输出数据存储寄时钟
      output    wire                shcp          ,    //移位寄存器的时钟输入
      output    wire                ds            ,    //串行数据输入
      output    wire                oe                 //输出使能信号
);

//wire    define
wire      [19:0]    data      ;    //数码管要显示的值
wire      [5:0]     point     ;    //小数点显示,高电平有效 top_seg_595
wire                seg_en    ;    //数码管使能信号，高电平有效
wire                sign      ;    //符号位，高电平显示负号
//-------------data_gen_inst--------------
data_gen       data_gen_inst
(
      .sys_clk        (sys_clk    ),    //系统时钟，频率 50MHz
      .sys_rst_n      (sys_rst_n),    //复位信号，低电平有效

      .data           (data       ),    //数码管要显示的值
      .point          (point      ),    //小数点显示,高电平有效
      .seg_en         (seg_en     ),    //数码管使能信号，高电平有效
      .sign           (sign       )     //符号位，高电平显示负号
);

//-------------seg7_dynamic_inst--------------
seg_595_dynamic       seg_595_dynamic_inst
(
      .sys_clk       (sys_clk     ),    //系统时钟，频率 50MHz
      .sys_rst_n     (sys_rst_n ),    //复位信号，低有效
      .data          (data        ),    //数码管要显示的值
      .point         (point       ),    //小数点显示,高电平有效
      .seg_en        (seg_en      ),    //数码管使能信号，高电平有效
      .sign          (sign        ),    //符号位，高电平显示负号

      .stcp          (stcp        ),    //输出数据存储寄时钟
      .shcp          (shcp        ),    //移位寄存器的时钟输入
```

```
    .ds             (ds             ),    //串行数据输入
    .oe             (oe             )     //输出使能信号
);
Endmodule
```

四． ModelSim 仿真环境、测试结果及分析

上电复位后，交通灯控制模块进入初始相位，倒计时装载为绿灯 60s。系统以 1s 为单位更新倒计时：每到 1s 使能脉冲到来，倒计时数值减 1，当倒计时减到 0 时，状态机切换到下一相位并重新装载对应时长。

在相位切换过程中，灯色变化与倒计时严格同步：

绿灯相位（60s）：主通行方向为绿灯，另一方向为红灯；数码管显示当前相位剩余秒数（从 60 开始递减）。

黄灯相位（5s）：绿灯结束后进入黄灯过渡，相位持续 5s；数码管倒计时从 5 递减到 0，提示即将切换到下一相位（保证过渡安全）。

实验过程中观察到：倒计时变化连续、每秒跳变一次；灯色切换时刻与倒计时归零时刻一致，没有出现提前/滞后或闪烁异常，说明 1s 节拍、状态机与显示链路工作正常。

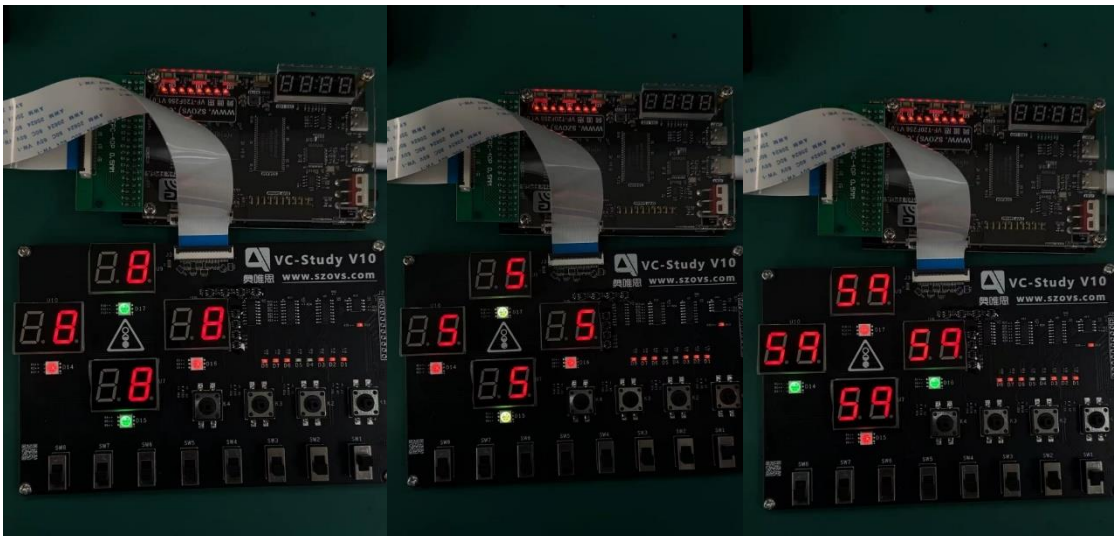


图-测试结果

从图中可见，开发板数码管能够稳定显示倒计时数值（如 59、5、8 等），且数值随时间按秒递减，说明计时器与动态显示链路工作正常；同时交通灯在不同相位下能正确切换并显示主路与支路的红、绿、黄灯状态：绿灯相位对应主路通行并进行 60s 倒计时，切换到黄灯过渡相位时倒计时装载为 5s 并递减，进

一步表明主路与支路的灯色输出与计时、相位切换同步一致，系统整体运行正常。

五．实验总结

本实验围绕十字路口交通灯控制及倒计时显示这一典型数字系统展开，完成了一个从需求分析、模块划分、代码实现、功能仿真到板级验证的完整设计闭环。系统上电复位后进入预设的初始相位，倒计时器按参数装载为绿灯 60 秒，并在精准的 1 秒使能脉冲控制下逐秒递减；当倒计时归零时，状态机立即切换至下一相位，并重新装载对应的持续时间（如黄灯 5 秒），从而实现“装载—递减—归零切换—再装载”的稳定循环。实验现象清晰可靠：数码管能够连续、稳定地显示当前相位的剩余时间（如 59、58…5、4…），且相位切换的瞬间，交通灯灯色变化与倒计时归零严格同步，未出现任何提前、滞后或闪烁异常。在灯色表现上，主干道与支干道的红、绿、黄灯组合完全符合设计预期——绿灯相位保障主路通行、支路禁行，黄灯相位则提供明确的过渡提示，确保相位转换的安全性。整个系统最终呈现出“控制逻辑正确、显示输出稳定、时序严格一致”的特点，这不仅验证了设计的功能完备性，也体现了其在工程实现上的鲁棒性与可靠性。

通过本次实验，我对课堂所授的同步时序设计与有限状态机（FSM）核心思想有了显著深入的理解。课堂上强调的所有时序行为都应在同一时钟边沿下完成更新，在本实验中得到了具体落实：倒计时寄存器仅在 1 秒使能脉冲有效时才进行更新，而状态机的相位切换也严格遵循“倒计时归零条件成立且时钟上升沿触发”的同步机制，这种“条件判断＋时序同步”的设计方法，从根本上避免了竞争冒险和逻辑毛刺，保证了系统行为的确定性。在时基生成部分，我更加具体地认识到“分频计数器”在实际工程中的意义：从 50MHz 系统时钟衍生出精确的 1Hz 节拍，并非简单的延时操作，而是一个涉及计数器位宽、终值判断与复位时序的严谨过程；任何细节偏差都可能导致节拍失真，进而引发倒计时快慢不一或相位切换紊乱等宏观问题。此外，FSM 的“状态定义—转移条件—输出逻辑”三要素在本实验中得到了生动体现：状态对应交通灯的四个相位，转移条件由倒计时归零及外部传感器信号共同决定，输出逻辑则直接映射为灯色控制信号与下一相位的倒计时初值。相比于课堂上抽象的状态转移图，本次实践将状态机从“图形描述”转化为真正可在硬件中稳定运行的“时序电路”，使我深刻意识到：状

态机设计不仅需关注逻辑正确性，还需综合考虑状态寄存器的更新时机、输出译码方式（Moore 型或 Mealy 型）以及其与计时器、传感器等其他模块的协同交互关系。

本次实验还进一步强化了我对“控制—显示—驱动”系统级设计思维的认识。在课堂学习中，各功能模块常被独立讲解，然而实际工程中，控制层产生的信号必须通过可靠的显示链路才能被人机界面所感知，而显示链路又必须严格匹配硬件驱动的电气与时序特性。本实验所涉及的二进制至 BCD 转换、数码管动态扫描以及基于 74HC595 的串行扩展输出等环节，使我深切体会到：控制模块输出一个数值，并不等同于“用户能看到正确且稳定的数值”；其间还需经过扫描频率的合理设置、段码与位码的时序配合、串行移位与锁存信号的精准同步等多重处理，否则极易出现显示残影、字符错乱或亮度不均等问题。在调试过程中，我认识到功能仿真虽能验证状态机与倒计时逻辑的正确性，但板级实测才能暴露显示刷新、信号极性、驱动时序等与硬件密切相关的实际问题；只有将仿真与实测相结合，才能完成真正的闭环验证，确保系统在真实环境中可靠运行。总而言之，本实验不仅让我掌握了交通灯控制系统的基本实现方法，更促使我将课堂中所学的时序逻辑、有限状态机、分频计数与显示驱动等多方面知识，有机整合为一个可实际运行的系统框架。

实验题目： 异步串口通信 UART 实验

学　　院： 自动化学院

班　　级： 电信 2303 班

指导教师： 张俊

学　　号： 8207230322

姓　　名： 樊博文

一. 实验内容及要求

实验 5 异步串口通信 UART 实验（Quartus + Intel 实验箱）

内容：

1. 工程目录: test5，输入第十章中异步串口通信UART的实现代码并拷贝至该目录，并作如下修改：将接受模块u_rec.v中r_START状态下的rec_readyInH=HI行删除；将u_xmit.v中WORD_LEN改成WORD_LEN-1。
2. 创建ModelSim仿真工程，测试模块文件为testbench.v；通过波形观察RXD与TXD上的传输是否正确，确认消息窗口是否显示：

```
VSIM 8> run -all
# TESTBENCH TX: 00 11 22 33 44 55 66 77 88 99 aa
#
# TESTBENCH RX: 00 11 22 33 44 55 66 77 88 99 aa
#
# Break in Module test6 at C:/work/test/test6/sim/test6.v line 57
```

3. 创建quartus工程test_uart，并添加除testbench.v以外的所有设计文件，其中test_uart.v为顶层模块文件，mp_fake.v为模拟处理器模块。
4. 参考下表管脚说明使用PIN Editor或者Assignment Editor手动分配管脚。

对应 FPGA 管脚名	端口名称
PIN_G1	clk
PIN_K21	rstn(用按键 1 作为复位)
PIN_N7(COM2),PIN_V1(COM1)	TXD

PIN_U2(COM2),PIN_P6(COM1)	RXD
PIN_D22	led0
PIN_D21	led1

5. 编译设计：右击test_uart.v，选择“Set as top-level entity”,点击Processing > Start Compilation或对应全程编译按键,查看编译报告。
6. 将EP4C40核心板右上角的JTAG用十芯排线和USB下载器的接口连接起来。将I/O扩展接口JC5用二十芯排线和LED流水灯实验区的JC2连接，将I/O扩展接口JC4用二十芯排线和拔码开关实验区的JC_K1连接。打开电源。
7. 将设计文件编程至FPGA：点击Tools > Programer或对应编程按键，在弹出窗口中检查Hardware Setup右侧栏中是否有对应电缆设置，如果没有点击Hardware Setup进行设置；勾选设计文件test_uart.sof的Program/Configure列，点击Start进行下载；
8. 观察LED1是否点亮（表示发送模块空闲），LED0是否熄灭（表示无新的接受数据）。
9. 连接RS232电缆，运行串口通信.exe,设置串口（设备管理器中可查找串口号），波特率4800，数据位8， 校验位无，停止位2，不勾选校验，勾选HEX、自动换行、显示间隔等；点击“打开串口”，在发送栏输入数据“00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F”,点击发送，观察接受窗口是否能够接受到相同数据。

二. 实验原理及方法

1.实验原理

UART（通用异步收发器）是一种基于串行通信的接口技术，广泛用于数据的双向传输。它通过两根数据线实现通信：一根用于接收数据，另一根用于发送数据。采用异步传输模式，意味着发送端和接收端并不共享时钟信号，从而避免了需要同步时钟的复杂性。UART 协议遵循 RS-232C 标准，由美国电子工业联合会（EIA）与贝尔公司于 1969 年联合开发，至今仍被广泛应用于各种通信设备之间的数据交换。

串行通信指的是在数据传输过程中，信息通过单一数据线路逐位传输。与并

行通信相比，串行通信的传输速度较低，但由于使用的数据线数量少，其在远距离通信中的成本优势显著。此外，串行通信的电缆结构较为简单，且较易实现设备间的长距离通信，因此在现代电子设备中得到了广泛应用。

在系统架构中，CPU 通常通过并行接口与内部设备进行通信，而外设则多通过串行接口与计算机进行数据交换。串行通信接口的核心功能是实现数据的转换与传输。在数据发送过程中，串行接口会将来自 CPU 的并行数据转换为串行数据，依次通过发送移位寄存器逐位传输至外设；而在数据接收时，接口则将外设发送的串行数据逐位接收，并组装为并行数据，再通过数据输入寄存器传递给 CPU 处理。UART 接口的硬件结构如图所示。

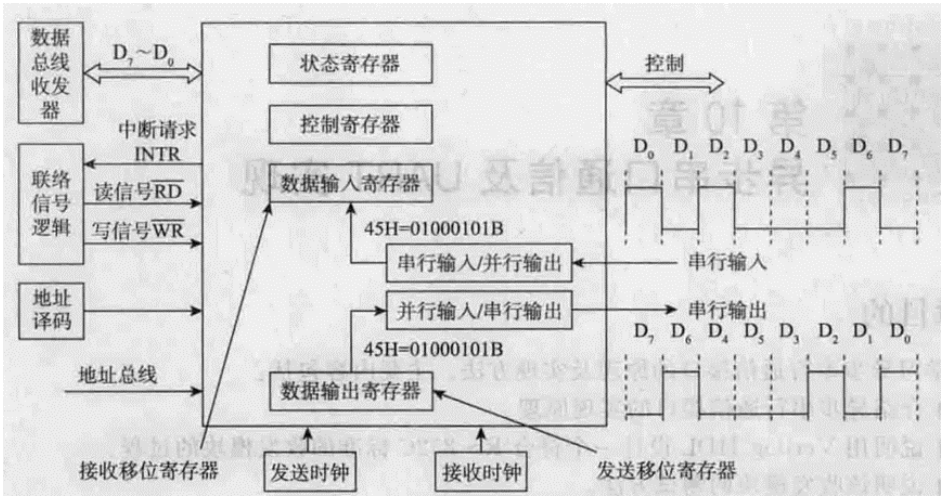


图-UART 接口的组成框图

在接收数据的过程中，外部设备通过串行线路将数据逐位送入接收移位寄存器。当接收移位寄存器中所有数据位接收完成后，数据将被传输到数据输入寄存器，供 CPU 并行读取。接收移位寄存器的移位速率由接收时钟控制，确保数据的逐位准确接收。

数据输出过程则是 CPU 将并行数据送入数据输出寄存器，随后数据通过发送移位寄存器逐位传送至外设。发送移位寄存器的移位速率由发送时钟决定，确保数据按顺序传输。

控制寄存器保存了与接口操作相关的控制信息，CPU 可通过修改控制寄存器来调整串行通信接口的工作方式。状态寄存器通过多个状态位提供实时的通信状态反馈，如数据输入寄存器是否已满、数据输出寄存器是否为空，或是否发生了数据传输错误等情况。

在 UART 的串行通信过程中，每一帧数据（即一个数据包）通常包含四个部分：起始位、数据位、可选的校验位和停止位。起始位通常为逻辑“0”，用于标识数据传输的开始；数据位的长度可根据需要调整，通常为 6 到 8 位；校验位用于数据的完整性检查，通常选择奇校验或偶校验；停止位用于标示数据包的结束，其逻辑状态为“1”。如图所示，每一帧数据按这种格式传输。

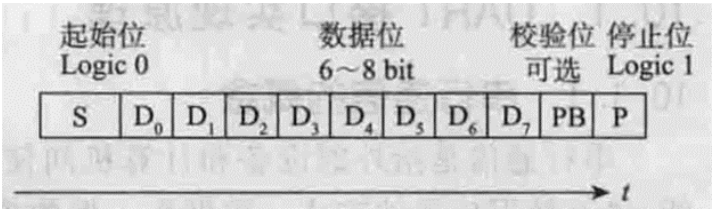


图-基本 UART 包的格式

值得注意的是，UART 协议是异步的，即发送端与接收端不共享时钟信号。为了保证数据传输的准确性，发送方和接收方在传输开始前必须协商一致的波特率。在实际通信中，发送和接收波特率通常是相同的。通信双方通过预定的波特率同步各自的内部时钟，确保数据位的正确采样。接收端通过精确的时钟采样技术，在每个数据位的中心点进行采样，从而避免时序误差的发生。

UART 协议的核心特点之一是其较高的时钟采样频率。以常见的 16450 系列 UART 为例，其内部时钟频率为数据传输波特率的 16 倍，这使得接收端能够在波特率的 1/16 周期内进行过采样，确保了数据接收的高精度。

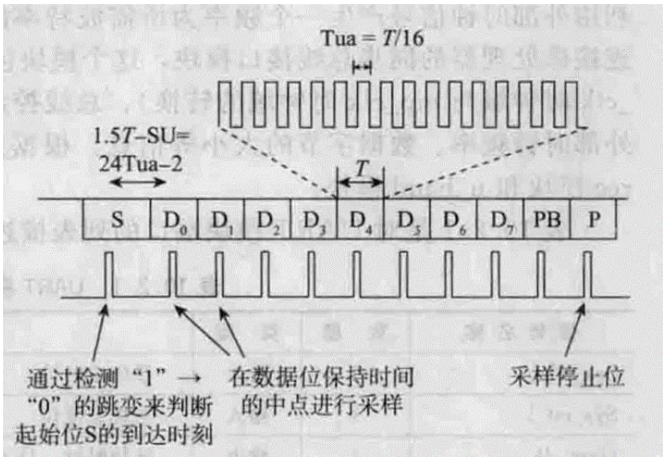


图-UART 接收模块的数据取样点

如图展示了 UART 通信中数据帧的时序关系及其采样逻辑。数据帧的组成包括起始位（S）、数据位（D_0-D_7）、可选的校验位（PB）和停止位（P）。起始位用于标志数据传输的开始，电平为逻辑“0”；数据位表示传输的数据内容，一般为 6 到 8 位，这里为 8 位；校验位用于校验传输数据的正确性，可选择奇校验、

偶校验或无校验；停止位标志数据帧结束，电平为逻辑“1”，长度可为 1 位、1.5 位或 2 位。

接收模块在数据位的中心位置对其进行采样，以避免由于噪声或信号抖动导致的错误判断。对于停止位的采样，接收模块在其逻辑“1”的状态下完成检测，以确保数据帧传输结束后，通信线路回到空闲状态（逻辑“1”）。

通过以上设计，UART 协议在异步通信中实现了数据传输的高精度与可靠性。

2.实验方法

采用分层和模块化的设计思想，可以将 UART 接口设计划分为两个主要部分：微处理器接口模块和 UART 收发接口模块。其中，UART 收发接口模块进一步分为三个子模块：发送模块(u_xmit)、接收模块(u_rec)以及波特率变换模块(baud)。

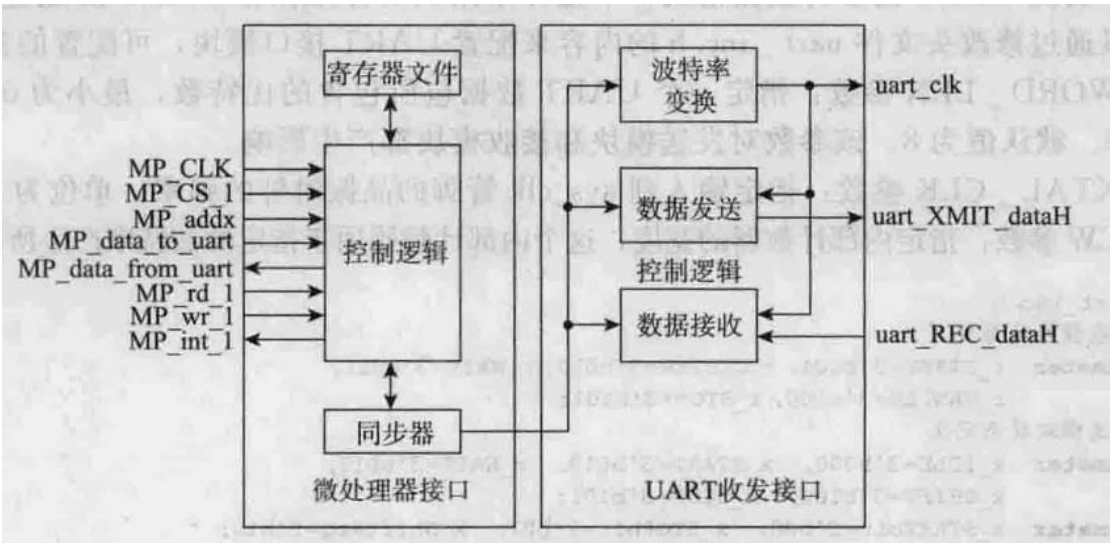


图-UART 接口的顶层功能设计框图

如图展示了 UART 接口的顶层功能设计框图，其 Verilog HDL 描述由多个文件组成，包括：uart.v、u_xmit.v、u_rec.v、mp_int.v、baud.v 和配置文件 uart_inc.h。其中，uart.v 是顶层模块，负责实例化和连接所有子模块。这个顶层模块本身并不包含任何逻辑设计，仅用于将各功能模块组织为一个整体。

模块 u_xmit.v 负责实现异步发送功能。它的内部结构包括状态机、串行化器和相关逻辑电路，用于将并行数据转换为串行数据并逐位发送。模块 u_rec.v 则负责异步接收功能，内含一个两级同步器、状态机以及相关逻辑，用于将接收到的串行数据恢复为并行数据，同时处理必要的同步操作。

波特率变换模块 baud.v 通过外部时钟信号生成一个频率为目标波特率 16 倍的内部“波特时钟”，以支持发送和接收过程中的高精度采样。该模块是 UART

通信中实现数据传输时序控制的核心。

模块 mp_int.v 作为微处理器的同步总线接口模块，负责处理 UART 与微处理器之间的连接与通信。该模块包括寄存器文件、时钟域间的同步装置（实现 uart_clk 时钟域和 mp_clk 时钟域的转换）以及总线控制逻辑，确保数据在不同时钟域之间稳定传输。

配置文件 uart_inc.h 为整个设计提供必要的参数配置，其中包含外部时钟频率、数据字节大小等信息。这些配置信息直接用于生成与目标应用相匹配的时钟信号，并分别传递到接收模块（u_rec）和波特率变换模块（baud），确保整个系统能够按照预定的通信协议正确运行。

三．代码设计（说明修改优化部分）

1.将接受模块 u_rec.v 中 r_START 状态下的 rec_readyInH=HI 行删除；

```
//u_rec.v
module u_rec (
        sys_rst_l,
        sys_clk,
        uart_dataH,
        rec_dataH,
        rec_readyH
        );

        `include "uart_inc.h"
        input sys_rst_l;
        input sys_clk;
        input uart_dataH;
        output [7:0] rec_dataH;
        output rec_readyH;
        reg [2:0] next_state,state;
        reg rec_datH, rec_datSyncH;
        reg [3:0] bitCell_cntrH;
        reg cntr_resetH;
        reg[7:0] par_dataH;
        reg shiftH;
        reg[3:0] recd_bitCntrH;
        reg countH;
        reg rstCountH;
        reg rec_readyH;
        reg rec_readyInH;
```

```
    wire [7:0] rec_dataH;
assign rec_dataH=par_dataH;
//两级同步，消除亚稳态
always@ (posedge sys_clk or negedge sys_rst_l)
    if (~sys_rst_l) begin
        rec_datSyncH <= 1;
        rec_datH <= 1;
    end
    else begin
        rec_datSyncH <= uart_dataH;
        rec_datH <= rec_datSyncH;
    end

// 位元计数器
always@ (posedge sys_clk or negedge sys_rst_l)
    if (~sys_rst_l)
        bitCell_cntrH <= 0;
    else if (cntr_resetH)
        bitCell_cntrH <= 0;
    else
        bitCell_cntrH <= bitCell_cntrH+ 1;
//串行输入移位寄存器
always@ (posedge sys_clk or negedge sys_rst_l)
    if (~sys_rst_l)
        par_dataH <= 0;
    else if(shiftH) begin
        par_dataH[6:0] <= par_dataH[7:1];
        par_dataH[7]    <= rec_datH;
end
// 已接受位数计数器
always@ (posedge sys_clk or negedge sys_rst_l)
    if (~sys_rst_l)
        recd_bitCntrH <= 0;
    else if(countH)
        recd_bitCntrH <= recd_bitCntrH+1;
    else if(rstCountH)
        recd_bitCntrH <= 0;
// 状态机
always@ (posedge sys_clk or negedge sys_rst_l)
    if (~sys_rst_l)
        state <= r_START;
    else
    state <= next_state;
```

```
always@ (state or rec_datH or bitCell_cntrH or recd_bitCntrH )
begin
    case (state)
        r_START:begin
            if (~ rec_datH) begin
                next_state = r_CENTER;
                cntr_resetH =HI;
                shiftH      =LO;
                countH      =LO;
                rstCountH   =LO;
                rec_readyInH=LO;
            end
            else begin
                next_state          = r_START;
                rstCountH           = HI;
                cntr_resetH =HI;
                shiftH      =LO;
                countH      =LO;
                rec_readyInH=LO;
//              rec_readyInH  = HI;
            end
        end
        r_CENTER:begin
            if (bitCell_cntrH == 4'h4) begin
                if (~ rec_datH) begin
                    next_state = r_WAIT;
                    cntr_resetH =HI;
                    shiftH      =LO;
                    countH      =LO;
                    rstCountH   =LO;
                    rec_readyInH=LO;
                end
                else begin
                    next_state = r_START;
                    cntr_resetH =HI;
                    shiftH      =LO;
                    countH      =LO;
                    rstCountH   =LO;
                    rec_readyInH=LO;
                end
            end
            else begin
                next_state    = r_CENTER;
                cntr_resetH   = LO;
```

```
            shiftH          =LO;
            countH          =LO;
            rstCountH       =LO;
            rec_readyInH=LO;
        end
end
r_WAIT:begin
        if(bitCell_cntrH == 4'hE) begin
            if (recd_bitCntrH == WORD_LEN) begin
                next_state = r_STOP;
                cntr_resetH =HI;
                shiftH          =LO;
                countH          =LO;
                rstCountH       =LO;
                rec_readyInH=LO;
            end
            else begin
                next_state = r_SAMPLE;
                cntr_resetH =HI;
                shiftH          =LO;
                countH          =LO;
                rstCountH       =LO;
                rec_readyInH=LO;
            end
        end
        else begin
            next_state      = r_WAIT;
            cntr_resetH     = LO;
            shiftH          =LO;
            countH          =LO;
            rstCountH       =LO;
            rec_readyInH=LO;
        end
end
r_SAMPLE:begin
        shiftH          = HI;
        countH          = HI;
        next_state      = r_WAIT;
        cntr_resetH =HI;
        rstCountH       =LO;
        rec_readyInH=LO;
end
r_STOP:begin
        next_state          = r_START;
```

```
                rec_readyInH  = HI;
                cntr_resetH =HI;
                shiftH          =LO;
                countH           =LO;
                rstCountH      =LO;
            end
            default:begin
                next_state          = 3'bxxx;
                cntr_resetH         = X;
                shiftH              = X;
                countH              = X;
                rstCountH           = X;
                rec_readyInH  = X;
            end
        endcase
    end
    //向异时钟域传送时，必须保存后输出，消除危险路径
    always@ (posedge sys_clk or negedge sys_rst_l)
            if (~sys_rst_l)
                rec_readyH <= 0;
            else
                rec_readyH <= rec_readyInH;
endmodule
```

2.将 u_xmit.v 中 WORD_LEN 改成 WORD_LEN-1

```
//u_xmit.v
module u_xmit(
    sys_clk,
    sys_rst_l,
    uart_xmitH,
    xmitH,
    xmit_dataH,
    xmit_doneH
);

    `include "uart_inc.h"
    input sys_clk;
    input sys_rst_l;
    output uart_xmitH;
    input xmitH;
    input[7:0] xmit_dataH;
    output xmit_doneH;
    reg [2:0] next_state, state;
    reg load_shiftRegH;
```

```
reg shiftEnaH;
reg [4:0] bitCell_cntrH;
reg countEnaH;
reg [7:0] xmit_ShiftRegH;
reg [3:0] bitCountH;
reg rst_bitCountH;
reg ena_bitCountH;
reg [1:0] xmitDataSelH;
reg uart_xmitH;
reg xmit_doneInH;
reg xmit_doneH;

// 多路选择器
always@ (xmit_ShiftRegH or xmitDataSelH)
    case (xmitDataSelH)
        x_STARTbit: uart_xmitH = LO;
        x_STOPbit:   uart_xmitH = HI;
        x_ShiftReg:     uart_xmitH = xmit_ShiftRegH[0];
        default:       uart_xmitH = X;
    endcase
// 位元计数器
always@ (posedge sys_clk or negedge sys_rst_l)
    if (~sys_rst_l)
        bitCell_cntrH <= 0;
    else if (countEnaH)
        bitCell_cntrH <= bitCell_cntrH+ 1;
    else
        bitCell_cntrH <= 0;
//移位寄存器
always@ (posedge sys_clk or negedge sys_rst_l)
    if (~sys_rst_l)
        xmit_ShiftRegH <= 0;
    else   if (load_shiftRegH)
        xmit_ShiftRegH <= xmit_dataH;
    else if (shiftEnaH) begin
        xmit_ShiftRegH[6:0]      <= xmit_ShiftRegH[7:1];
        xmit_ShiftRegH[7]        <= HI;
    end
    else
        xmit_ShiftRegH <= xmit_ShiftRegH;

//已发送位计数器
always@ (posedge sys_clk or negedge sys_rst_l)
    if (~sys_rst_l)
```

```
            bitCountH <= 0;
        else if (rst_bitCountH)
            bitCountH <= 0;
        else if (ena_bitCountH)
            bitCountH <= bitCountH+ 1;
// 状态机
always@ (posedge sys_clk or negedge sys_rst_l)
        if (~sys_rst_l)
            state <= x_IDLE;
        else
            state <= next_state;

always@ (state or xmitH or bitCell_cntrH or bitCountH)
begin
        next_state              = state;
        load_shiftRegH      = LO;
        countEnaH               = LO;
        shiftEnaH               = LO;
        rst_bitCountH       = LO;
        ena_bitCountH       = LO;
        xmitDataSelH        = x_STOPbit;
        xmit_doneInH        = LO;
         case (state)
            x_IDLE:begin
                if (xmitH) begin
                    next_state = x_START;
                    load_shiftRegH = HI;
                end
                else begin
                    next_state = x_IDLE;
                    rst_bitCountH = HI;
                    xmit_doneInH = HI;
                end
            end
            x_START:begin
                xmitDataSelH = x_STARTbit;
                if (bitCell_cntrH == 4'hf)
                    next_state = x_WAIT;
                else begin
                    next_state      = x_START;
                    countEnaH       = HI;
                end
            end
            x_WAIT:begin
```

```
                xmitDataSelH = x_ShiftReg;
                if (bitCell_cntrH == 4'hE) begin
                    if(bitCountH == WORD_LEN - 1)
                        next_state = x_STOP;
                    else begin
                        next_state          = x_SHIFT;
                        ena_bitCountH       = HI;
                    end
                end
                else begin
                    next_state      = x_WAIT;
                    countEnaH       = HI;
                end
            end
            x_SHIFT:begin
                xmitDataSelH  = x_ShiftReg;
                next_state          = x_WAIT;
                shiftEnaH       = HI;
            end
            x_STOP:begin
                xmitDataSelH = x_STOPbit;
                if (bitCell_cntrH == 4'hF) begin
                    next_state = x_IDLE;
                    xmit_doneInH = HI;
                end
                else begin
                    next_state = x_STOP;
                    countEnaH=HI;
                end
            end
            default:begin
                next_state          = 3'bxxx;
                load_shiftRegH    = X;
                countEnaH           = X;
                shiftEnaH           = X;
                rst_bitCountH     = X;
                ena_bitCountH     = X;
                xmitDataSelH      = 2'bxx;
                xmit_doneInH      = X;
            end
        endcase
end
    //向异时钟域传送时，必须保存后输出，消除危险路径
always@ (posedge sys_clk or negedge sys_rst_l)
```

```
        if (~sys_rst_l)
            xmit_doneH <= 0;
        else
            xmit_doneH <= xmit_doneInH;
endmodule
```

3. UART 顶层封装模块-UART.v

```
//UART.v
module UART(
    sys_rst_l,
    sys_clk,
    uart_clk,
    mp_clk,
    mp_cs_l,
    mp_addx,
    mp_data_to_uart,
    mp_data_from_uart,
    mp_rd_l,
    mp_wr_l,
    mp_int_l,
    uart_XMIT_dataH,
    uart_REC_dataH,
    stat_rec_dataH,
    stat_xmit_emptyH
);
    `include "uart_inc.h"
    input sys_rst_l;
    input sys_clk;
    output uart_clk;
    input mp_clk;
    input mp_cs_l;
    input [2:0] mp_addx;
    input [7:0] mp_data_to_uart;
    output [7:0] mp_data_from_uart;
    wire xmit_busy_rstH;
    input   mp_rd_l;
    input mp_wr_l;
    output mp_int_l;
    output   uart_XMIT_dataH;
    input   uart_REC_dataH;
    output   stat_rec_dataH;
    output   stat_xmit_emptyH;
    wire uart_clk;
```

```
wire [15:0]baud_rate_div;
wire xmit_start_pulseH;
wire xmit_doneH;
wire [7:0]reg_xmit_dat;
wire xmit_busyH;
wire [7:0]rec_dataH;
wire rec_readyH;
wire stat_rec_dataH;
wire stat_xmit_emptyH;
mp_int iMP_INT(
	.sys_rst_l(sys_rst_l),
	.uart_clk(uart_clk),
	.mp_clk(mp_clk),

	.mp_cs_l(mp_cs_l),
	.mp_addx(mp_addx),
	.mp_data_from_uart(mp_data_from_uart),
	.mp_data_to_uart(mp_data_to_uart),
	.mp_rd_l(mp_rd_l),
	.mp_wr_l(mp_wr_l),
	.mp_int_l(mp_int_l),

	.baud_rate_div(baud_rate_div),

	.start_pulseH(xmit_start_pulseH),
	.xmit_doneH(xmit_doneH),
	.reg_xmit_dat(reg_xmit_dat),
	.xmit_busyH(xmit_busyH),
	.xmit_busy_rstH(xmit_busy_rstH),
	.rec_dataH(rec_dataH),
	.rec_readyH(rec_readyH),

	.stat_rec_dataH(stat_rec_dataH),
	.stat_xmit_emptyH(stat_xmit_emptyH)
	);
u_xmit    iXMIT(
	.sys_clk(uart_clk),
	.sys_rst_l(sys_rst_l),
	.uart_xmitH(uart_XMIT_dataH),
	.xmitH(xmit_start_pulseH),
	.xmit_dataH(reg_xmit_dat),
	.xmit_doneH(xmit_doneH)		);

u_rec iRECEIVER(
```

```
        .sys_rst_l(sys_rst_l),
        .sys_clk(uart_clk),
        .uart_dataH(uart_REC_dataH),
        .rec_dataH(rec_dataH),
        .rec_readyH(rec_readyH)          );
    baud iBAUD(
        .sys_clk(sys_clk),
        .sys_rst_l(sys_rst_l),
        .baud_clk(uart_clk),
        .baud_rate_div(baud_rate_div)
    );

endmodule
```

4. 模拟微处理器模块 mp_fake.v 的功能设计

//mp_fake.v

```
module mp_fake(mp_clk, sys_rst_l, mp_cs_l, mp_addx, mp_data_to_uart, mp_data_from_uart,
mp_rd_l, mp_wr_l, mp_int_l);
    input               mp_clk;
    input               sys_rst_l;
    output              mp_cs_l;
    output [2:0]    mp_addx;
    output [7:0]    mp_data_to_uart;
    input [7:0]     mp_data_from_uart;
    output              mp_rd_l;
    output              mp_wr_l;
    input               mp_int_l;

    `include "uart_inc.h"

    parameter       IDLE = 3'h0,
                        W_BAUD_LO = 3'h1,
                        W_BAUD_HI = 3'h2,
                        W_INT_ENA = 3'h3,
                        WAIT = 3'h4,
                        R_DAT = 3'h5,
                        W_DAT = 3'h6,
                        CLR_INT = 3'h7;
    parameter           B_DIV = 16'h0145;
    reg [2:0]       c_state_reg, n_state;
    reg                 mp_cs_l, mp_rd_l, mp_wr_l;
    reg [2:0]       mp_addx;
    reg [7:0]       mp_data_to_uart;
```

```
reg                     save_data;
reg [7:0]       data_reg;

always@(posedge mp_clk or negedge sys_rst_l)
begin
    if (!sys_rst_l)
        c_state_reg <= IDLE;
    else
        c_state_reg <= n_state;
end

always@(c_state_reg or mp_data_from_uart or data_reg)
begin
    case(c_state_reg)
        IDLE : begin
            n_state = W_BAUD_LO;
            mp_cs_l = 1'b1;
            mp_rd_l = 1'b1;
            mp_wr_l = 1'b1;
            mp_addx = 3'h0;
            mp_data_to_uart = 8'h00;
            save_data = 1'b0;
        end
        W_BAUD_LO : begin
            n_state = W_BAUD_HI;
            mp_cs_l = 1'b0;
            mp_rd_l = 1'b1;
            mp_wr_l = 1'b0;
            mp_addx = SEL_BAUD_REG_LO;
            mp_data_to_uart = B_DIV[7:0];
            save_data = 1'b0;
        end
        W_BAUD_HI : begin
            n_state = W_INT_ENA;
            mp_cs_l = 1'b0;
            mp_rd_l = 1'b1;
            mp_wr_l = 1'b0;
            mp_addx = SEL_BAUD_REG_HI;
            mp_data_to_uart = B_DIV[15:8];
            save_data = 1'b0;
        end
        W_INT_ENA : begin
            n_state = WAIT;
            mp_cs_l = 1'b0;
```

```
        mp_rd_l = 1'b1;
        mp_wr_l = 1'b0;
        mp_addx = SEL_INT_ENA_REG;
        mp_data_to_uart = 8'h03;
        save_data = 1'b0;
end
WAIT : begin
        if (mp_data_from_uart[1:0] == 2'b11)
                n_state = R_DAT;
        else
                n_state = WAIT;
        mp_cs_l = 1'b0;
        mp_rd_l = 1'b0;
        mp_wr_l = 1'b1;
        mp_addx = SEL_INT_STAT_REG;
        mp_data_to_uart = 8'h00;
        save_data = 1'b0;
end
R_DAT : begin
        n_state = W_DAT;
        mp_cs_l = 1'b0;
        mp_rd_l = 1'b0;
        mp_wr_l = 1'b1;
        mp_addx = SEL_REC_DAT;
        mp_data_to_uart = 8'h00;
        save_data = 1'b1;
end
W_DAT : begin
        n_state = CLR_INT;
        mp_cs_l = 1'b0;
        mp_rd_l = 1'b1;
        mp_wr_l = 1'b0;
        mp_addx = SEL_XMIT_DAT;
        mp_data_to_uart = data_reg;
        save_data = 1'b0;
end
CLR_INT : begin
        n_state = WAIT;
        mp_cs_l = 1'b0;
        mp_rd_l = 1'b1;
        mp_wr_l = 1'b0;
        mp_addx = SEL_INT_STAT_REG;
        mp_data_to_uart = 8'h00;
        save_data = 1'b0;
```

```
                end
                default : begin
                    n_state = WAIT;
                    mp_cs_l = 1'b1;
                    mp_rd_l = 1'b1;
                    mp_wr_l = 1'b1;
                    mp_addx = 8'h00;
                    mp_data_to_uart = 8'h00;
                    save_data = 1'b0;
                end
            endcase
        end

        always@(posedge mp_clk or negedge sys_rst_l)
        begin
            if (!sys_rst_l)
                data_reg <= 8'h00;
            else if (save_data)
                data_reg <= mp_data_from_uart;
        end

endmodule
```

5. 中断与寄存器接口模块

```
//MP_INT.v
module mp_int(
    //系统信号
    sys_rst_l, uart_clk, mp_clk,
    //微处理器接口
    mp_cs_l, mp_addx, mp_data_to_uart, mp_data_from_uart, mp_rd_l, mp_wr_l, mp_int_l,
    //波特率变换模块接口
    baud_rate_div,
    //UART 发送模块接口
    start_pulseH, xmit_doneH, reg_xmit_dat, xmit_busyH,
    // UART 接收模块接口
    rec_dataH, rec_readyH, xmit_busy_rstH, stat_xmit_emptyH, stat_rec_dataH,
    //  调试接口
    wr_done_pulseH, sel_xmit_datH
    );

    `include "uart_inc.h"
    input       sys_rst_l;
```

```
input       mp_clk;
input       uart_clk;
input       mp_cs_l;
input       [2:0]mp_addx;
input       mp_rd_l;
input       mp_wr_l;
input       [7:0]mp_data_to_uart;
output      [7:0]mp_data_from_uart;
output      [15:0]baud_rate_div;
output      start_pulseH;
input       xmit_doneH;
output      xmit_busyH;
output      [7:0]reg_xmit_dat;
input       [7:0]rec_dataH;
input       rec_readyH;
output      mp_int_l;
output      xmit_busy_rstH;
output      stat_xmit_emptyH;
output      stat_rec_dataH;
output      wr_done_pulseH;
output      sel_xmit_datH;
reg         [7:0]reg_baud_rate_div_lo_in;
reg         [7:0]reg_baud_rate_div_hi;
reg         [7:0]reg_baud_rate_div_lo;
reg         [7:0]reg_xmit_dat;
reg         [7:0]reg_int_ena;
reg         xmit_busyH;
reg         [7:0]mp_date_out;
reg         stat_rec_dataH;
reg         stat_xmit_emptyH;
reg         rec_new_dataH;
wire        [7:0]mp_data;
wire        sel_baud_regLoH;
wire        sel_baud_regHiH;
wire        sel_xmit_datH;
wire        sel_int_maskH;
wire        wr_done_pulseH;
wire        xmit_busyH_sync;
wire        start_pulseH;
wire        xmit_doneH_sync;
wire        [15:0]baud_rate_div;
wire        xmit_busy_rstL;
wire        rec_readyH_sync;
wire        rec_ready_pulseH;
```

```
wire        sel_int_stat_regH;

assign mp_data                  = mp_data_to_uart;
assign mp_data_from_uart = mp_date_out;
assign baud_rate_div        =                   //波特率输出
            {reg_baud_rate_div_hi,reg_baud_rate_div_lo};
assign sel_baud_regLoH     = ~ mp_cs_l &    //波特率低位
            (mp_addx==SEL_BAUD_REG_LO);
assign sel_baud_regHiH     = ~ mp_cs_l &    //波特率高位
            (mp_addx==SEL_BAUD_REG_HI);
assign sel_xmit_datH        = ~ mp_cs_l &    //  发送数据
            (mp_addx==SEL_XMIT_DAT);
assign sel_int_maskH        = ~ mp_cs_l &    //中断使能
            (mp_addx==SEL_INT_ENA_REG);
assign sel_int_stat_regH = ~ mp_cs_l &    //  中断状态
            (mp_addx==SEL_INT_STAT_REG);

//写寄存器,微处理器可访问的寄存器文件
always@ (posedge mp_clk or negedge sys_rst_l)
begin
      if (~sys_rst_l) begin
            reg_baud_rate_div_lo       <= 0;
            reg_baud_rate_div_hi       <= 0;
            reg_xmit_dat                 <= 0;
            reg_int_ena                   <= 0;
      end
      else begin
            if (~mp_wr_l)
            begin
                  if (sel_baud_regLoH)
                        reg_baud_rate_div_lo_in <= mp_data;
                  if (sel_baud_regHiH)
                  begin
                        reg_baud_rate_div_hi <= mp_data;
                        reg_baud_rate_div_lo <= reg_baud_rate_div_lo_in;
                  end
                  if (sel_xmit_datH)
                        reg_xmit_dat <= mp_data;
                  if (sel_int_maskH)
                        reg_int_ena <= mp_data;
            end
      end
end
//读寄存器，输出多路选择
```

```
    always@ (mp_addx or rec_dataH or reg_baud_rate_div_lo or reg_baud_rate_div_hi
                or stat_rec_dataH or stat_xmit_emptyH or reg_int_ena or rec_new_dataH or
xmit_busyH)
    begin
        case (mp_addx)
            SEL_REC_DAT:
                mp_date_out = rec_dataH;
            SEL_BAUD_REG_LO:
                mp_date_out = reg_baud_rate_div_lo;
            SEL_BAUD_REG_HI:
                mp_date_out = reg_baud_rate_div_hi;
            SEL_STAT_REG:
                mp_date_out = {rec_new_dataH, xmit_busyH};
            SEL_INT_STAT_REG:
                mp_date_out = {stat_rec_dataH, stat_xmit_emptyH};
            SEL_INT_ENA_REG:
                mp_date_out = reg_int_ena;
            default:
                mp_date_out = 8'hxx;
        endcase
    end
    // 产生中断
    assign mp_int_l = ~((stat_xmit_emptyH & reg_int_ena[0]) |
                                    (stat_rec_dataH    & reg_int_ena[1]));
    // 接受数据有效，对应 STAT_REG[1]
always@ (posedge mp_clk or negedge sys_rst_l)
    begin
        if (~sys_rst_l)
            rec_new_dataH <= 0;
        else if (rec_ready_pulseH)    //接受到一个字节数据
            rec_new_dataH <= 1'b1;
        else if (sel_xmit_datH & mp_rd_l)   //读取了接受到的数据
            rec_new_dataH <= 1'b0;
    end
    //接受完成,对应 INT_STAT_REG[1:0]
    always@ (posedge mp_clk or negedge sys_rst_l)
    begin
        if(~sys_rst_l)
            stat_xmit_emptyH   <= 1'b1;
        else begin
            if (sel_int_stat_regH & ~ mp_wr_l) //  通过写清除
                stat_xmit_emptyH <=mp_data[0];
            else if (xmit_busy_rstH)                    //完成一个字节发送
                stat_xmit_emptyH <= 1'b1;
```

```
            else if (sel_xmit_datH & ~ mp_wr_l) //写入待发送数据
                stat_xmit_emptyH <= 0;
        end
    end
    //发送空闲,对应 INT_STAT_REG[1:0]
    always@ (posedge mp_clk or negedge sys_rst_l)
    begin
        if(~sys_rst_l)
            stat_rec_dataH      <= 1'b0;        //不同结构分开写（需修改）
        else begin
            if (sel_int_stat_regH & ~ mp_wr_l) //  通过写清除
                stat_rec_dataH <= mp_data[1];
            else if (rec_ready_pulseH) //接受到一个字节数据，产生中断
                stat_rec_dataH <= 1'b1;
        end
    end



    assign xmit_busy_rstL = ~xmit_busy_rstH; //发送结束

    one_shot OS_WR (
        .clk_in(mp_clk),
        .sys_rst_l(sys_rst_l),
        .d(mp_wr_l),
        .q(wr_done_pulseH) );

//  wire rst = sys_rst_l & xmit_busy_rstL; //多余

    //向发送模块启动发送,对应 STAT_REG[0]
//  always@ (posedge mp_clk or negedge xmit_busy_rstL)          //不推荐内部信号,做异步
复位，应改成同步复位。
    always@ (posedge mp_clk)   //内部信号同步复位
        if (~ xmit_busy_rstL)
            xmit_busyH <= 0;
//      else if (wr_done_pulseH & sel_xmit_datH)
        else if (~mp_wr_l & sel_xmit_datH)
            xmit_busyH <= 1;

    sync SYNC_WR (
        .clk_in(uart_clk),                      // 2 级同步
        .sys_rst_l(xmit_busy_rstL),
        .d(xmit_busyH),
        .q(xmit_busyH_sync) );
```

```
    one_shot OS_WR2 (                    //检测上升沿
        .clk_in(uart_clk),
        .sys_rst_l(sys_rst_l),
        .d(xmit_busyH_sync),
        .q(start_pulseH) );
    //  接受发送结束信号
    sync SYNC_WR2 (
        .clk_in(mp_clk),                        // 2 级同步
        .sys_rst_l(sys_rst_l),
        .d(xmit_doneH),
        .q(xmit_doneH_sync) );

    one_shot OS_WR3 (                    //检测上升沿
        .clk_in(mp_clk),
        .sys_rst_l(sys_rst_l),
        .d(xmit_doneH_sync),
        .q(xmit_busy_rstH) );
    //接受到接受数据信号
    sync SYNC_RECDAT(
        .clk_in(mp_clk),                            // 2 级同步
        .sys_rst_l(sys_rst_l),
        .d(rec_readyH),
        .q(rec_readyH_sync) );

    one_shot OS_RECDAT(                  //检测上升沿
        .clk_in(mp_clk),
        .sys_rst_l(sys_rst_l),
        .d(rec_readyH_sync),
        .q(rec_ready_pulseH) );

endmodule
```

6. 波特率与时序支撑模块设计的设计与实现

//baud.v

```
module mp_fake(mp_clk, sys_rst_l, mp_cs_l, mp_addx, mp_data_to_uart, mp_data_from_uart,
mp_rd_l, mp_wr_l, mp_int_l);
    input           mp_clk;
    input           sys_rst_l;
    output          mp_cs_l;
    output [2:0]    mp_addx;
    output [7:0]    mp_data_to_uart;
    input [7:0]     mp_data_from_uart;
    output          mp_rd_l;
```

```
output          mp_wr_l;
input           mp_int_l;

`include "uart_inc.h"

parameter   IDLE = 3'h0,
            W_BAUD_LO = 3'h1,
            W_BAUD_HI = 3'h2,
            W_INT_ENA = 3'h3,
            WAIT = 3'h4,
            R_DAT = 3'h5,
            W_DAT = 3'h6,
            CLR_INT = 3'h7;
parameter   B_DIV = 16'h0145;
reg [2:0]   c_state_reg, n_state;
reg         mp_cs_l, mp_rd_l, mp_wr_l;
reg [2:0]   mp_addx;
reg [7:0]   mp_data_to_uart;
reg         save_data;
reg [7:0]   data_reg;

always@(posedge mp_clk or negedge sys_rst_l)
begin
    if (!sys_rst_l)
        c_state_reg <= IDLE;
    else
        c_state_reg <= n_state;
end

always@(c_state_reg or mp_data_from_uart or data_reg)
begin
    case(c_state_reg)
        IDLE : begin
            n_state = W_BAUD_LO;
            mp_cs_l = 1'b1;
            mp_rd_l = 1'b1;
            mp_wr_l = 1'b1;
            mp_addx = 3'h0;
            mp_data_to_uart = 8'h00;
            save_data = 1'b0;
        end
        W_BAUD_LO : begin
            n_state = W_BAUD_HI;
            mp_cs_l = 1'b0;
```

```
        mp_rd_l = 1'b1;
        mp_wr_l = 1'b0;
        mp_addx = SEL_BAUD_REG_LO;
        mp_data_to_uart = B_DIV[7:0];
        save_data = 1'b0;
end
W_BAUD_HI : begin
        n_state = W_INT_ENA;
        mp_cs_l = 1'b0;
        mp_rd_l = 1'b1;
        mp_wr_l = 1'b0;
        mp_addx = SEL_BAUD_REG_HI;
        mp_data_to_uart = B_DIV[15:8];
        save_data = 1'b0;
end
W_INT_ENA : begin
        n_state = WAIT;
        mp_cs_l = 1'b0;
        mp_rd_l = 1'b1;
        mp_wr_l = 1'b0;
        mp_addx = SEL_INT_ENA_REG;
        mp_data_to_uart = 8'h03;
        save_data = 1'b0;
end
WAIT : begin
        if (mp_data_from_uart[1:0] == 2'b11)
                n_state = R_DAT;
        else
                n_state = WAIT;
        mp_cs_l = 1'b0;
        mp_rd_l = 1'b0;
        mp_wr_l = 1'b1;
        mp_addx = SEL_INT_STAT_REG;
        mp_data_to_uart = 8'h00;
        save_data = 1'b0;
end
R_DAT : begin
        n_state = W_DAT;
        mp_cs_l = 1'b0;
        mp_rd_l = 1'b0;
        mp_wr_l = 1'b1;
        mp_addx = SEL_REC_DAT;
        mp_data_to_uart = 8'h00;
        save_data = 1'b1;
```

```
                end
                W_DAT : begin
                        n_state = CLR_INT;
                        mp_cs_l = 1'b0;
                        mp_rd_l = 1'b1;
                        mp_wr_l = 1'b0;
                        mp_addx = SEL_XMIT_DAT;
                        mp_data_to_uart = data_reg;
                        save_data = 1'b0;
                end
                CLR_INT : begin
                        n_state = WAIT;
                        mp_cs_l = 1'b0;
                        mp_rd_l = 1'b1;
                        mp_wr_l = 1'b0;
                        mp_addx = SEL_INT_STAT_REG;
                        mp_data_to_uart = 8'h00;
                        save_data = 1'b0;
                end
                default : begin
                        n_state = WAIT;
                        mp_cs_l = 1'b1;
                        mp_rd_l = 1'b1;
                        mp_wr_l = 1'b1;
                        mp_addx = 8'h00;
                        mp_data_to_uart = 8'h00;
                        save_data = 1'b0;
                end
        endcase
    end

    always@(posedge mp_clk or negedge sys_rst_l)
    begin
        if (!sys_rst_l)
                data_reg <= 8'h00;
        else if (save_data)
                data_reg <= mp_data_from_uart;
    end

endmodule

//ONE_SHOT.v
module one_shot (sys_rst_l, clk_in, d, q);
    input    sys_rst_l;
```

```
    input    clk_in;
    input    d;
    output q;
    reg       d_del;
    always@ (posedge clk_in or negedge sys_rst_l)
        if (~sys_rst_l)
            d_del <= 1'b1;
        else d_del <= ~d;

    assign q=d & d_del;

endmodule

//SYNC.V
module sync (clk_in,sys_rst_l, d, q);
    input    clk_in,sys_rst_l, d;
    output q;
    reg    pipe1, pipe2;
    always@ (posedge clk_in or negedge sys_rst_l)
         if (~sys_rst_l) begin
            pipe1 <= 0;
            pipe2 <= 0;
        end
        else begin
            pipe1 <= d;
            pipe2 <= pipe1;
        end
    assign q= pipe2;
endmodule
```

7.test_uart.v

```
module test_uart(clk, rstn, TXD, RXD, led0, led1);
    input            clk;
    input            rstn;
    output    TXD;
    input            RXD;
    output    led0;
    output    led1;

    wire        mp_cs_l, mp_rd_l, mp_wr_l, mp_int_l;
    wire [2:0] mp_addx;
    wire [7:0] mp_data_from_uart, mp_data_to_uart, stat_rec_dataH, stat_xmit_emptyH;
```

```
    UART uart(
        .sys_rst_l(~rstn),
        .sys_clk(clk),
        .uart_clk(),
        .mp_clk(clk),
        .mp_cs_l(mp_cs_l),
        .mp_addx(mp_addx),
        .mp_data_to_uart(mp_data_to_uart),
        .mp_data_from_uart(mp_data_from_uart),
        .mp_rd_l(mp_rd_l),
        .mp_wr_l(mp_wr_l),
        .mp_int_l(mp_int_l),
        .uart_XMIT_dataH(TXD),
        .uart_REC_dataH(RXD),
        .stat_rec_dataH(stat_rec_dataH),
        .stat_xmit_emptyH(stat_xmit_emptyH)
    );

    assign led0 = ~stat_rec_dataH;
    assign led1 = ~stat_xmit_emptyH;

    mp_fake  mp_fake(
        .mp_clk(clk),
        .sys_rst_l(~rstn),
        .mp_cs_l(mp_cs_l),
        .mp_addx(mp_addx),
        .mp_data_to_uart(mp_data_to_uart),
        .mp_data_from_uart(mp_data_from_uart),
        .mp_rd_l(mp_rd_l),
        .mp_wr_l(mp_wr_l),
        .mp_int_l(mp_int_l)
    );

endmodule
```

8.testbench.v

```
`timescale 1ns/1ns

module testbench;
    reg clk, rstn;
    reg RXD;
    wire TXD, led0, led1;
```

```
test_uart test_uart(
        .clk(clk),
        .rstn(rstn),
        .TXD(TXD),
        .RXD(RXD),
        .led0(led0),
        .led1(led1)
);

always begin
        clk = 1'b1;
        #10 clk = 1'b0;
        #10;
end

initial begin
        rstn = 1'b0;
        #100 rstn = 1'b1;
end

initial begin
        RXD = 1'b1;
        #100000;
        #208333 RXD = 1'b0;
        #208333 RXD = 1'b1;
        #208333 RXD = 1'b0;
        #208333 RXD = 1'b1;
        #208333 RXD = 1'b0;
        #208333 RXD = 1'b1;
        #208333 RXD = 1'b0;
        #208333 RXD = 1'b1;
        #208333 RXD = 1'b0;
        #208333 RXD = 1'b1;

        #208333 RXD = 1'b0;
        #208333 RXD = 1'b1;
        #208333 RXD = 1'b0;
        #208333 RXD = 1'b1;
        #208333 RXD = 1'b0;
        #208333 RXD = 1'b1;
        #208333 RXD = 1'b0;
        #208333 RXD = 1'b1;
        #208333 RXD = 1'b0;
        #208333 RXD = 1'b1;
```

```
            #(208333*10);
//          $stop;
        end
endmodule
//    // TESTBENCH TX
//    reg [7:0]    txdata[0:10];
//    integer      tx_i;
//    initial begin
//        RXD = 1'b1;
//     wait(rstn);
//        #100;
//
//     for (tx_i = 0;tx_i <= 10;tx_i = tx_i+1) begin
//          txdata[tx_i] = 8'h11*tx_i;
//          uart_txbyte(txdata[tx_i]);
//     end
//     $display("TESTBENCH TX: %x %x %x %x %x %x %x %x %x %x %x\n",
txdata[0],txdata[1],txdata[2],txdata[3],txdata[4],txdata[5],txdata[6],txdata[7],txdata[8],txdata[9],t
xdata[10]);
//
//    end
//
//    // TESTBENCH RX
//    reg [7:0]    rxdata[0:10];
//    integer      rx_i;
//    initial begin
//     wait(rstn);
//        #100;
//     for (rx_i = 0;rx_i <= 10;rx_i = rx_i+1) begin
//          rxdata[tx_i] = 8'h11*rx_i;
//          uart_rxbyte(rxdata[rx_i]);
//     end
//     $display("TESTBENCH RX: %x %x %x %x %x %x %x %x %x %x %x\n",
rxdata[0],rxdata[1],rxdata[2],rxdata[3],rxdata[4],rxdata[5],rxdata[6],rxdata[7],rxdata[8],rxdata[9],
rxdata[10]);
//
//     #1000;
//     $stop;
//    end
//
//     task uart_txbyte;
//          input          [7:0]        i_data;
//            integer                        i;
//
```

```
//          `ifdef PARITY_CHECK
//          integer                         one_cnt;
//          `endif
//    begin
//          `ifdef PARITY_CHECK
//          one_cnt = 0;
//          `endif
//
// //         $display("TESTBENCH TX: will send one byte : %x\n", i_data);
//
//          #100;
//          RXD = 1'b0;
//
//            for (i=7; i>=0; i=i-1) begin
//                #100;
//                RXD = i_data[i];
//
//                `ifdef PARITY_CHECK
//                if (i_data[i])
//                      one_cnt = one_cnt + 1;
//                `endif
//          end
//
//          `ifdef PARITY_CHECK
//          #208333;
//          if (one_cnt[0])
//                RXD = 1'b0;
//          else
//                RXD = 1'b1;
//          `endif
//
//          #100;
//          RXD = 1'b1;
//      end
//      endtask
//
//    task uart_rxbyte;
//          output          [7:0]           o_data;
//          integer                         i;
//          `ifdef PARITY_CHECK
//          integer                         one_cnt;
//          `endif
//    begin
//            wait(!TXD);
```

```
//          #100;
//
//
//            o_data = 8'h00;
//            for (i=7; i>=0; i=i-1) begin
//                #100;
//                  o_data[i] = TXD;
//                `ifdef PARITY_CHECK
//                if (TXD)
//                      one_cnt = one_cnt + 1;
//                `endif
//                #100;
//          end
//
//          `ifdef PARITY_CHECK
//          #100;
//            if (one_cnt[0] == TXD) begin
//                  $display("TESTBENCH RX: Odd bit is wrong!\n");
//                #100;
//                  $stop;
//            end
//          #100;
//          `endif
//
//          #100;
//            if (!TXD) begin
//                  $display("TESTBENCH RX: Stop bit is wrong!\n");
//                #100;
//                  $stop;
//            end
//          #100;
//
////          $display("TESTBENCH RX: Received one byte : %x\n", o_data);
//      end
//      endtask
//endmodule
```

四． ModelSim 仿真环境、测试结果及分析

Led0 为 1 时，表示处于熄灭状态，意味着当前没有新的接收数据；而 Led1 为 0 时，表示处于点亮状态，说明发送模块处于空闲状态。根据这两个状态，可以初步判断窗口具备发送和接收数据的功能，仿真结果如下图：

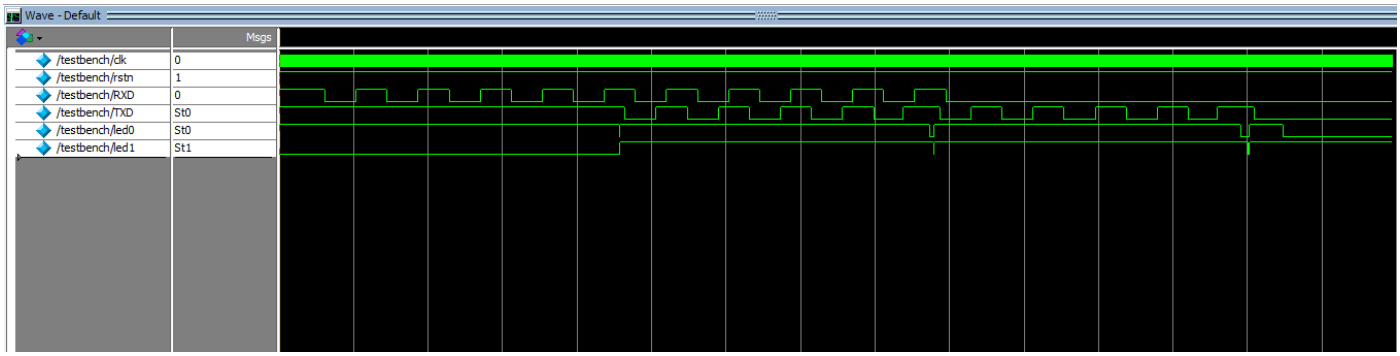


图-ModelSim 仿真

在 Testbench 文件的测试过程中，复位后，串口模块会先接收发送出的测试数据，随后再将接收到的数据重新发送出去。因此，在仿真结果中，RXD 与 TXD 的波形应该完全一致，且 RXD 的波形领先于 TXD。仿真波形如图所示，RXD 在前，TXD 在后，这表明仿真结果符合预期，验证了设计的正确性。

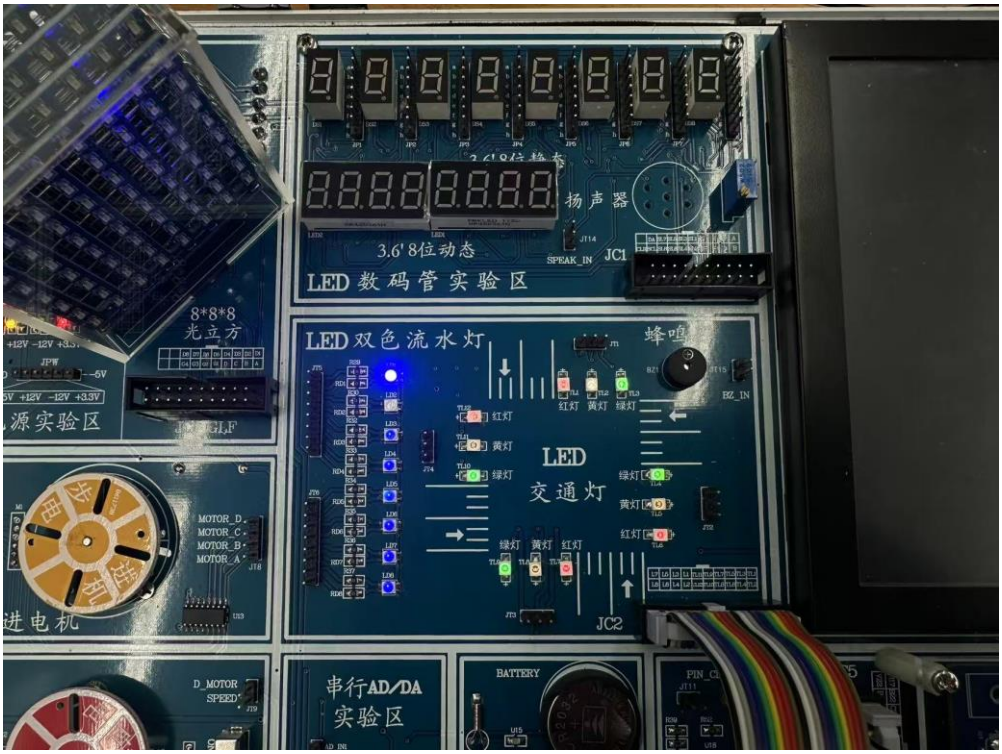


图-开发板测试

进行实物测试时，首先观察开发板指示灯：空闲状态下 LED1 保持点亮，表明发送模块可随时进入发送流程；LED0 保持熄灭，表明暂未产生新的接收完成事件，系统处于稳定等待接收状态。当串口接收到一帧有效数据后，接收状态会发生相应变化并驱动指示更新，整体表现与仿真结论一致，说明 UART 模块不仅在仿真中逻辑正确，在硬件上也能稳定工作。

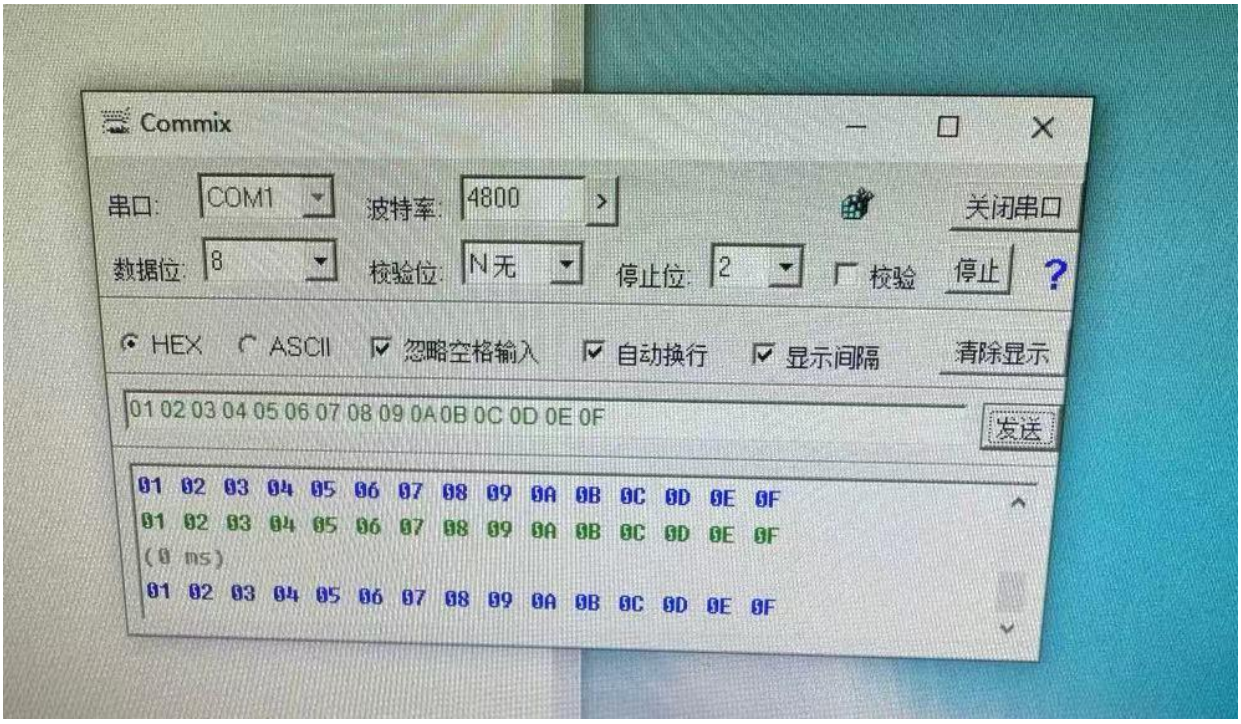


图-串口测试

随后连接 RS232 电缆并运行 串口通信.exe，在设备管理器中确认串口号后，将串口参数设置为：波特率 4800、数据位 8、校验位无、停止位 2，不勾选校验，同时勾选 HEX、自动换行、显示间隔等选项。点击“打开串口”后，在发送栏输入测试数据 “00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F”并发送；若接收窗口能够完整显示与发送区一致的十六进制序列，则说明 FPGA 端已正确完成数据接收与原样回传，串口参数匹配、收发链路通畅，通信过程稳定且数据一致性良好，满足实验要求。

本实验通过 ModelSim 仿真与板级实测对 UART 收发功能进行了完整验证：仿真中 RXD 与 TXD 波形帧结构一致且 RXD 领先于 TXD，证明先接收再回传的时序与控制逻辑正确；实物测试中 LED 指示灯状态与仿真结论一致，表明收发模块在硬件上运行稳定。进一步在串口通信软件中以 4800bps、8N2 参数发送固定 HEX 测试序列，并能在接收窗口得到完全一致的数据回显，说明串口参数配置正确、数据链路无误码、回环转发可靠。由此确认本 UART 设计满足实验要求，实现了稳定的异步串行通信收发与数据一致性验证。

五．实验总结

本实验成功实现并验证了 UART 异步串口通信的完整收发功能与数据回环机制，完成了从 ModelSim 仿真、板级硬件联调到上位机通信验证的全流程实践。系统在复位释放后，首先由接收模块通过 RXD 引脚捕获 Testbench 或上位机发送的测试数据，经过起始位检测、数据位采样与组帧后，生成有效的接收字节；随后发送模块将该字节重新按 UART 帧格式通过 TXD 引脚发出，从而形成“接收→处理→回传”的闭环测试。因此在仿真波形中可清晰观测到：RXD 与 TXD 的帧结构完全一致，且因接收处理带来的固有延迟，RXD 信号在时序上领先于 TXD；实际仿真结果与此完全吻合，证实了接收端的位采样、帧对齐以及发送端的移位控制等逻辑均正确无误。在板级测试阶段，指示灯状态为系统工作状态提供了直观指示：发送通道空闲时 LED1 保持常亮，表明发送模块随时可响应发送请求；无新接收完成事件时 LED0 保持熄灭，说明系统处于稳定的等待接收状态。进一步连接 RS232 串口线，借助上位机软件配置通信参数为 4800bps、8 位数据位、无校验、2 位停止位（8N2），并以 HEX 格式发送“00 01 … 0F”测试序列后，接收窗口能够完整、准确地回显出相同数据，充分证明了参数匹配正确、通信链路通畅、数据回环转发稳定可靠，系统已具备完整的 UART 串口通信能力。

本实验最为重要的收获在于将课堂上关于“异步通信”的理论概念转化为可实际运行的硬件逻辑。课堂中强调，UART 之所以能在无独立时钟线的情况下实现可靠通信，核心在于通信双方必须预先约定波特率，并由接收端基于该速率进行精确的位采样：起始位用于同步帧起始边界，随后在每个数据位的中心附近进行采样以抑制信号抖动，最终通过停止位恢复至空闲状态。通过本次实践，我深刻体会到：UART 协议看似简单，但其实现中任何一个时序环节的偏差都可能导致误码或帧错误。例如，若发送端位计数边界设置错误，会造成数据位多发或少发，从而使接收端帧对齐失败；若接收端在起始位检测阶段误置位 ready 信号，将导致上层逻辑误判“已有有效数据”，进而引发数据错读或重复读取。通过按实验要求对代码进行针对性修改（如删除接收状态机中起始检测阶段不合理的 ready 置位、调整发送端 WORD_LEN 计数边界），我对“控制信号产生时机”这一关键设计原则有了更为直观和深刻的理解：在同步时序系统中，ready、valid 等握手信号必须与数据稳定的时刻严格对齐，任何超前或滞后的置位都可能引发难以调试的边界错误。相比仅仅记忆 UART 的帧格式，真正保证系统正确运行的往

往是这些细致入微的时序约束与状态控制逻辑。

此外，本次实验也帮助我构建起更为系统化的数字系统验证与调试方法论。课堂上常强调“先仿真，后上板”的开发流程，而本实验进一步体现了“多重证据链”协同验证的重要性：第一层是 ModelSim 仿真波形，可精确观测 RXD/TXD 信号、状态机跳转、位计数器变化以及握手信号的时序关系，从逻辑层面确保设计正确;第二层是板级指示灯与运行现象,能快速判断系统是否处于预期状态(如空闲、接收中、发送中)，尤其适用于排查死锁、状态卡滞等硬件行为异常；第三层是上位机通信软件的数据一致性验证，通过真实串口环境检验参数配置、电平转换与数据链路的整体正确性。三者相互印证，能极大降低“仿真通过但上板失败”或“板级运行但数据异常”等问题的调试成本。通过这一过程，我更加理解了课堂中强调的“分层验证、逐步排查”工程思维：首先确认时钟与复位基础信号，继而验证帧结构与数据位宽，再检查握手信号与状态迁移时序，最终通过真实数据流完成端到端的系统级验证。综上所述，本实验不仅使我掌握了 UART 通信模块的具体实现技术，更将课堂理论知识转化为可操作、可调试的工程设计能力，为后续拓展支持校验位、可配置波特率、加入 FIFO 缓冲机制乃至实现更复杂串行通信协议奠定了坚实的实践基础。